Regret Is Freedom Lost

Michael Timothy Bennett m@michaeltimothybennett.com
School of Computing
Australian National University
Canberra, ACT 2601, Australia
ORCID 0000-0001-6895-8782

Abstract

How much of a reinforcement learning agent's regret comes from its memory, and how much from its policy? Its memory M represents histories by memory states. A policy must make the same prediction for the same task whenever histories share a memory state, so merging can incur irreducible loss. Together with avoidable errors in decisions, this is regret. Errors are weighted by task frequency and history importance. I prove that least normalised regret equals $K_\rho(M)$, the weight deleted to leave one correct prediction per memory state and task. Policies that predict the same past can diverge when predicting the future. Freedom of function measures how many futures remain open to a policy, while simplicity is a measure of its encoding. In encodings where freedom and simplicity are identical, $K_\rho(M)$ is weighted freedom lost. Under the corresponding independent prior over unseen futures, $e^{-K_\rho(M)}$ gives the probability of generalisation. Freedom selects trained memories using unlabelled target histories. On unseen tasks, it cut regret by 69% on Repeat Previous against choosing a memory by its regret on the task used for training. On a delayed cue, it cut regret by 58%. In that delayed-cue experiment, even random selection beat choosing by regret on the task used for training. This connects reinforcement learning with Stack Theory, a theory of optimal embodied learning.

Keywords: reinforcement learning, partial observability, model selection, freedom, Stack Theory

1 Introduction

When an agent compresses its history of observations and actions into a memory, it can lose information. Much of that loss can be harmless. However, in some cases the agent may no longer be able to tell apart histories for which correct actions differ. In reinforcement learning (RL), this loss is measured as regret, which is the value by which an agent under-performs the best it could possibly have done.

Here I show that regret is a special case of *freedom of function*, which has the advantage of differentiating between memories when regret cannot.[1] Freedom is a measure of how much more specific a policy can get, or alternatively how under-specified you might consider it to be. The freedom of a policy is the number of further commitments the agent could make, given what it has already committed to and the limitations imposed by its embodiment. Freedom is often described in terms of natural selection, where survival is contingent on an ability to cope with unseen demands. Under maximal ignorance about which demands the

1. Earlier work also referred to this quantity as *weakness* of constraints on function. I use freedom because it expresses what the quantity measures, while the notation w and the order $<_w$ are those of earlier work (Bennett, 2025).

agent will be subjected to in future, a policy with more freedom of function is more likely to allow the agent to *meet* those demands. Under both exchangeable and uniform priors,[2] maximising freedom is necessary to maximise adaptability (Bennett, 2025).

Typically, freedom is used for model or policy selection. However, policy freedom is affected by embodiment, and an agent's memory is part of its embodiment. So freedom can also be used to choose between memories. With this in mind, a prediction task specifies what the agent must predict from a history. Regret is scored relative to some *support*, which is a weighted set of history-task pairs. Pairs with the same memory state and the same task form a *group*. My argument is as follows:

1. Suppose predictions are stored as a table with one entry for each memory state and task. Then the least normalised regret of any policy equals the least weight that must be deleted from the support to leave one correct prediction in each group. This is the least weighted freedom a policy loses in a *degenerate* vocabulary, one in which every set of commitments is consistent, meaning they don't contradict (Theorem 3.3 and Corollary 3.6). It equals the weighted Bayes risk of the grouping, which is the least weighted loss when each group's prediction is chosen separately. The margin weight of the RL selection theorems (Nayebi, 2026) equals the prior weight of Stack Theory (Bennett, 2025). Of course, a language in which no two things contradict is entirely unrealistic, but the point of this is to show how freedom relates to Bayes risk and regret. In practice, the quantities will usually differ because freedom captures the limitations of embodiment.

2. In any practical scenario, the language will be non-degenerate. The agent uses a *decision rule*, which assigns a prediction to each memory state and task. When this rule is a linear layer, the predictions in every group depend on one weight vector, so some combinations of predictions are mutually exclusive[3]. The least regret among allowed decision rules is the least freedom lost by the predictions of one of those rules (Theorem 4.2). Bayes risk is the least regret when each group may have whichever prediction has the smaller loss. The least regret among allowed rules therefore equals the Bayes risk only if one of those rules assigns the prediction with the smaller loss in every group. In a recurrent network, two histories almost never share a hidden state, so each group contains a single history and every recurrent network has zero Bayes risk. With a linear layer, these networks differ only in the freedom lost (Remark 4.3).

3. At deployment, an agent may face tasks that it has never faced before. The memory most likely to support correct predictions for those unexpected tasks is that which allows for the greatest freedom, meaning the memory that incurs the least freedom lost when every task is equally likely (Proposition 4.4). In contrast, regret is computed only for *known tasks*, so it will fail to differentiate between two memories that differ only in their predictions of *unknown* tasks. This is a problem because generalisation to unknown tasks is arguably the whole point of AI (otherwise we'd just hard code answers).

2. A prior assigns probabilities to sets of future demands, given what an agent has observed. An exchangeable prior gives equal probability to sets containing the same number of demands. A uniform prior gives every set the same probability.
3. Which is what makes it non-degenerate.

4. An agent's actions often change what it observes, as when it walks through a maze. An *episode* is one run of an agent in its environment, from the first step to the last. Two agents that act differently produce different histories in their episodes. Each agent's regret is at least the freedom lost with its decision rules, measured on the histories from its own episodes (Theorem 4.5). To compare agents, I therefore run each one in the environment of deployment and record its histories, using no rewards. Those histories may hold too little information about the *hidden variable*, the variable that determines each correct action. For example, an agent can stop sampling a noisy cue before it knows which way to turn. To measure the loss from missing information, I compare each agent with a benchmark agent that observes the hidden variable and so always acts correctly. The loss against the benchmark splits into freedom never gained, freedom lost in memory and freedom left unused (Proposition 4.6). The first is loss from a history that does not determine the hidden variable, the second is loss from histories with different correct actions that share a memory state, and the third is loss from acting worse than the best decision rule for the memory.

5. A *teacher* is a model trained on labelled episodes from the *source*, the environment used for training. The teacher estimates every quantity above from unlabelled histories of the *target*, the environment of deployment. The memory chosen with these estimates falls short of the best memory by at most twice the teacher's error (Proposition 4.7).

Together, these points describe how to choose which trained agent to deploy, using no target rewards. A teacher estimates the freedom each agent loses on target histories, and *choosing by freedom* deploys the agent that loses the least. By points 1 to 4, an agent's regret on the target is at least the freedom it loses there. So choosing by freedom should deploy better agents than choosing by regret on known tasks, by performance where the agents trained, or by how confident the agents are. I test this prediction in four environments, under plans fixed in advance.

Agents faced new tasks at deployment in two settings. On POPGym's Repeat Previous (Morad et al., 2023), the agent observes one card per step and must name the suit of the kth most recent card. Here k is the *lag*. Agents trained on lag 4, and at deployment they had to predict suits at new lags. On the new lags, choosing by freedom cut the deployed agent's regret by 69% against choosing by regret on lag 4. Choosing by regret on lag 4 often meant deploying a threadbare transformer that held little but the card at that lag. In a delayed-cue experiment, agents faced two new prediction tasks at deployment. On those tasks, choosing by freedom cut regret by 58%. There, choosing by regret on the known task did worse than random choice.

In three further settings, the prediction tasks stayed the same, but agents chose how much to sense, the memory had to hold a cue for longer, or the agent had to walk further before it could use what it remembered. In those settings, I compared choosing by freedom with online validation, which tries each agent in the pool on rewarded target episodes. Online validation needed up to 64 episodes per agent to match choosing by freedom, which uses no rewards. Source validation ranks agents by their performance where they trained. It failed in at least one of those settings, as did every score of how confident an agent looks.

So what do these results actually mean outside RL? Stack Theory claims that to generalise, an embodied learner should maximise freedom of function, as opposed to simplicity of form

(Bennett, 2025). This is used to explain the adaptability of complex biological systems, even though complexity is often correlated with poor generalisation Bennett (2024b). RL measures failures to generalise as regret, so if regret is freedom lost, every RL result about regret is a result about freedom in one particular embodied language. Outside that language, regret and freedom differ, and each experiment here tests a case in which memories with equal regret differ in freedom. By extending freedom to RL, I demonstrate a practical use case for a relatively immature theoretical research program, and some empirical support for its claims about the role of weakness in selection.

2 Freedom

The definitions that follow are those of Bennett (2025).

2.1 Statements, extensions and freedom

Definition 2.1 (Environment, programs and vocabularies) *An* environment *is a nonempty set Φ of mutually exclusive states. A* program *is a set of states $p \subseteq \Phi$, the states in which it holds. A* vocabulary *is a finite set $\mathfrak{v}$ of programs.*

Definition 2.2 (Embodied language) *The* embodied language *of $\mathfrak{v}$ consists of all sets of its programs that can hold together in some state,*

$$L_{\mathfrak{v}} := \left\{ l \subseteq \mathfrak{v} \ : \ \textstyle\bigcap_{p \in l} p \neq \varnothing \right\}.$$

An element $l \in L_{\mathfrak{v}}$ is a statement*. It holds in the states $T(l) := \bigcap_{p \in l} p$, with $T(\varnothing) := \Phi$.*

A vocabulary stands for what a body can do, and a statement for one of the body's configurations. Most sets of programs are not statements, because a body cannot do everything in one configuration.

Definition 2.3 (Extension and freedom) *The* extension *of a statement $l \in L_{\mathfrak{v}}$ is the set of statements that contain it,* $\mathrm{Ext}(l) := \{\, y \in L_{\mathfrak{v}} \ : \ l \subseteq y \,\}$*. For a set $X \subseteq L_{\mathfrak{v}}$, the extension is the union of its members' extensions,* $\mathrm{Ext}(X) := \bigcup_{x \in X} \mathrm{Ext}(x)$*. The* freedom *of l is the number of statements in its extension, $w(l) := |\mathrm{Ext}(l)|$. The ordering $l_1 <_w l_2$ means $w(l_1) < w(l_2)$.*

An example. Take $\Phi = \{1, 2, 3, 4\}$ and three programs $a = \{1, 2\}$, $b = \{2, 3\}$, $c = \{3, 4\}$. Then $L_{\mathfrak{v}} = \{\varnothing, \{a\}, \{b\}, \{c\}, \{a, b\}, \{b, c\}\}$, because $a \cap c = \varnothing$. So $\mathrm{Ext}(\{a\}) = \{\{a\}, \{a, b\}\}$ and $\mathrm{Ext}(\{b\}) = \{\{b\}, \{a, b\}, \{b, c\}\}$, and hence $w(\{a, b\}) = 1 < w(\{a\}) = 2 < w(\{b\}) = 3$. Adding a program to a statement shrinks its extension, the set of its *completions*, so a statement that commits to more has fewer completions.

Interpretation. Freedom is the number of ways a statement can still be completed inside the agent's embodied language. It is a joint property of both the statement and the language in which it is made. Two statements that look equally complicated may differ in their freedom of function, because the sizes of their extensions depend on the language.

2.2 Tasks, generalisation and the value of freedom

Definition 2.4 (Task and correct policy) *A* $\mathfrak{v}$-task *is a pair* $\alpha = \langle I_\alpha, O_\alpha \rangle$. *Here* $I_\alpha \subseteq L_\mathfrak{v}$ *is its set of inputs, and* $O_\alpha \subseteq \mathrm{Ext}(I_\alpha)$ *is its set of correct outputs. A* policy *is a statement* $\pi \in L_\mathfrak{v}$. *It is* correct *for* α *when* $\mathrm{Ext}(I_\alpha) \cap \mathrm{Ext}(\pi) = O_\alpha$. *The set of policies correct for* α *is* Π_α. *Task* α *is a* child *of* ω *(*$\alpha \sqsubset \omega$*) when* $I_\alpha \subsetneq I_\omega$ *and* $O_\alpha \subseteq O_\omega$.

A child task represents what the learner has observed, like a training set. The parent task is the task from which the child is extracted, like a full benchmark dataset including training, test and validation data. So the learner picks a policy correct on the child and hopes it is correct on the parent. The set of *unseen* outputs is $U := L_\mathfrak{v} \setminus \mathrm{Ext}(I_\alpha)$. Suppose the parent's correct outputs are $O_\omega = O_\alpha \cup S$ for a random set $S \subseteq U$ of required outputs, which I call *demands*. A correct policy survives when, and only when, it rules out no output in S, that is when $S \subseteq \mathrm{Ext}(\pi) \cap U$.

Theorem 2.5 (Generalisation is P-weighted freedom) *Here* P *is the distribution of* S *given* α. *For every* $\pi \in \Pi_\alpha$,

$$\Pr(\pi \in \Pi_\omega \mid \alpha) = w_P(\pi) := P\big(S \subseteq \mathrm{Ext}(\pi) \cap U\big).$$

If P *is uniform on the subsets of* U, *then* $w_P(\pi) = 2^{|\mathrm{Ext}(\pi)\cap U| - |U|}$. *Under this prior, the correct policies with the most freedom on* U *are the most likely to generalise, so the Sufficiency and Necessity theorems of Bennett (2025) follow. If* P *is exchangeable under permutations of* U, *then* $w_P(\pi)$ *is a nondecreasing function of* $|\mathrm{Ext}(\pi) \cap U|$.

Proof The equality restates the survival condition. Here $A := \mathrm{Ext}(\pi) \cap U$ is the set of unseen outputs that the policy permits. Under the uniform prior, $2^{|A|}$ of the $2^{|U|}$ equally likely subsets of U lie inside A, giving the stated probability. If $A \subseteq A'$, then $\{S \subseteq A\} \subseteq \{S \subseteq A'\}$, so w_P is nondecreasing as A grows. Two subsets of U of the same size are images of each other under some permutation of U. Under exchangeability, they therefore have the same probability of containing S. Thus $P(S \subseteq A)$ depends on A only through $|A|$. Together with the preceding inclusion argument, this gives the claimed nondecreasing dependence on size. ■

When unseen outputs have unequal prior probabilities, the same argument holds with a weight on each unseen output.

Proposition 2.6 (Under independent priors, freedom is weighted) *Suppose each unseen output* $u_i \in U$ *becomes required independently with probability* $r_i \in [0, 1)$. *Then for every* $\pi \in \Pi_\alpha$,

$$\log \Pr(\pi \in \Pi_\omega \mid \alpha) = - \sum_{u_i \in U \setminus \mathrm{Ext}(\pi)} w_i, \qquad w_i := -\log(1 - r_i).$$

So maximising the probability of generalising is maximising the weighted freedom $W(\pi) := \sum_{u_i \in \mathrm{Ext}(\pi) \cap U} w_i$.

Proof The policy survives when no output that it prohibits turns out to be required. By independence, this event has probability $\prod_{u_i \in U \setminus \mathrm{Ext}(\pi)} (1 - r_i)$. Its logarithm is the stated negative sum of weights. The total weight over U is fixed, so minimising the weight of prohibited outputs maximises $W(\pi)$. ■

Interpretation. A policy fails to generalise when it prohibits outputs needed to fulfil demands that the world later imposes on the learner. Weighted freedom is the total weight of the demands that a policy still allows for, and the weight of each demand rises with the chance it will be imposed. Once the memory is fixed, some demands are out of reach, whatever the policy predicts. Every result in this paper is about those demands that are out of reach.

Remark 2.7 (Freedom is not simplicity) *Preferring the correct policy with the most freedom differs from preferring the shortest description. Description length is a matter of form, while freedom is a matter of function. The former depends on the encoding. A policy with the shortest description can sometimes have more freedom, and sometimes less freedom, in which case it generalises worse (Bennett, 2025). A vocabulary is* degenerate *when every set of its programs can be true at the same time. This represents a body that cannot represent mutual exclusivity, which would rule out anything like limbs or memory or logical* ***nand*** *gates (the latter being the most important for computation). In such a vocabulary, the completions of a statement l are the unions of l with each set of the other programs, so $w(l) = 2^{|\mathfrak{v}|-|l|}$ and preferring the most freedom is preferring the fewest programs. Nothing below depends on the length of a memory or a decision rule.*

3 In a degenerate vocabulary, regret is freedom lost

Regret depends, in part, on the choice of policy. A better choice of policy can avoid a great deal of regret. Of course, that doesn't mean you can eliminate regret. Even the best choice of policy may result in unavoidable regret, because some of the regret is built into the structure of memory. Hence, I will show that unavoidable regret with a fixed memory is freedom lost. The policies you can express are constrained by your memory, which is part of your embodiment. I will prove this first in RL terms, and then in Stack-Theoretic terms, facilitating a bridge between the two. The RL half follows Nayebi (2026), whose selection theorems show what an agent with low regret must hold in memory. In Nayebi's setting, the agent bets on one of two possible outcomes after observing a history, and the histories and prediction tasks are drawn jointly from a fixed distribution. For the rest, I draw on the literature on partially observable Markov decision processes (POMDPs) and state abstraction (Kaelbling et al., 1998; Li et al., 2006; Sutton and Barto, 2018).

3.1 Setup

A *history* h is the agent's interaction record up to a decision point. A *prediction task* T is to predict an outcome $Z_T \in \{0, 1\}$ about the world. After history h, the conditional probability of outcome 1 is $p_T(h) := \Pr(Z_T = 1 \mid h)$. The *support* is a finite list of history-task pairs $x_i = (h_i, T_i)$ for $i = 1, \ldots, n$. I call each pair a *point*. Each point has probability $\mu_i > 0$,

and these probabilities sum to one. At point i, the conditional probability is $p_i := p_{T_i}(h_i)$. Its distance from an even chance is the *margin* $m_i := |p_i - \frac{1}{2}|$. The best prediction is $y_i := \mathbf{1}\{p_i \geq \frac{1}{2}\}$.

A *memory* M is a function on histories. It is the structure of the agent's memory, and it determines which histories the agent can discriminate betwixt. The memory state $M(h)$ is the content of that memory after history h. Two points are in the same *group* when they have the same task and their histories have the same memory state. I write $i \sim_M j$ when points i and j are in the same group. The set of groups is $\mathcal{C}_M$. A policy *uses only* M when it assigns to each group one probability $q_C \in [0, 1]$ of predicting 1 on every point in the group.

Suppose a policy predicts 1 at point i with probability q. Its prediction is then correct with probability $V_i = q\, p_i + (1 - q)(1 - p_i)$. Under the best policy defined on h_i itself, the prediction is right with probability $V_i^\star = \max\{p_i, 1 - p_i\}$. The loss of a wrong prediction depends on the rewards of the deployment. I use three *loss weights* $\ell_i \geq 0$.

1. Normalised regret $\delta_i = 1 - V_i/V_i^\star$ (Nayebi, 2026) has weight $\rho_i := 2m_i/(\frac{1}{2} + m_i)$.
2. Success lost, $V_i^\star - V_i$, has weight $2m_i = |2p_i - 1|$.
3. Return lost has weight $s_i|2p_i - 1|$, where s_i is the extra return that a right prediction earns over a wrong prediction.

Lemma 3.1 (Loss at one point) *A policy that uses only M predicts 1 with probability q_C in the group $C \ni i$. For any loss weight above, its loss on point i is then*

$$\ell_i\Big((1 - q_C)\mathbf{1}\{y_i = 1\} + q_C\mathbf{1}\{y_i = 0\}\Big).$$

The proof is in Section A.

Interpretation. At each point, the policy's prediction differs from the best prediction with some probability. The loss at that point is this probability, multiplied by the loss weight. Weight approaches zero when the outcome is a coin flip (max entropy), and is large when an outcome is a near certainty.

3.2 Regret is weighted Bayes risk, and weighted forgetting

For a group C, the total weight of points whose best prediction is 1 is $A_C := \sum_{i \in C,\, y_i = 1} \mu_i \ell_i$. The total weight of points whose best prediction is 0 is $B_C := \sum_{i \in C,\, y_i = 0} \mu_i \ell_i$. Choosing the prediction with greater weight minimises the group's loss.

Definition 3.2 (Weighted forgetting cost) *Suppose I delete points from the support until each group's remaining points have the same best prediction. The least total weight of the deleted points is the weighted forgetting cost of M. With B denoting the set of deleted point indices, this cost is*

$$K_\ell(M) := \min\Big\{\textstyle\sum_{i \in B} \mu_i \ell_i \;\Big|\; B \subseteq \{1, \ldots, n\},\ y \text{ is constant on } C \setminus B \text{ for every } C \in \mathcal{C}_M\Big\}.$$

With the normalised-regret weight $\ell_i = \rho_i$, the cost is K_ρ. With the success weight $\ell_i = 2m_i$, the cost is K_Δ. With the unit weight $\ell_i = 1$, the cost is K_1.

Theorem 3.3 (Least regret equals weighted forgetting) *For every memory M and every loss weight above,*

$$\inf_{\pi \text{ uses only } M} \sum_{i=1}^{n} \mu_i \operatorname{loss}_i(\pi) \;=\; \sum_{C \in \mathcal{C}_M} \min\{A_C, B_C\} \;=\; K_\ell(M).$$

With $\ell = \rho$, the left side is the least average normalised regret of Nayebi (2026).

Proof By Lemma 3.1, a policy with probability q_C loses $A_C(1-q_C) + B_C q_C$ on group C. This loss has constant slope in q_C, so its least value on $[0,1]$ is $\min\{A_C, B_C\}$. A policy that uses only M can choose its prediction probability separately for each group, so adding these minima gives the first equality. For y to be constant among the points left in a group, deletion must remove all its $y=1$ points or all its $y=0$ points. The least weight that must be deleted is therefore $\min\{A_C, B_C\}$. The sum of these deleted weights over groups gives the second equality. ■

Interpretation. In a statistician's terms, the middle expression is the weighted Bayes risk of grouping points by memory state and task. The last expression is the same number interpreted as weighted forgetting. With only M to go on, a policy must assign the same prediction to every point in a group. Of course, if the points in a group differ in their best predictions, some of them will be assigned suboptimal predictions as a consequence of being grouped under M. I split each group into two parts. Those for whom the best prediction is 1, and those for whom the best prediction is 0. The larger the part, the greater its weight. Then the best policy predicts 1 if that part weighs more, and 0 if the 0 part weighs more. Then the points in the lighter part are discarded. Alas, one prediction per group is a Procrustean rule, and it incurs loss equal to the weight of the discarded points. This loss comes from the memory. To push regret below $K_\ell(M)$, the agent must change what it remembers. Once the groups are formed, computing $K_\ell(M)$ requires $O(n)$ time.

Example 3.4 (Two groups) *Take three points under one task with $\mu_i = \frac{1}{3}$ and $p = (0.9, 0.1, 0.7)$, so $y = (1,0,1)$ and $\rho = (\frac{8}{9}, \frac{8}{9}, \frac{4}{7})$. Let M be a memory under which h_1 and h_2 share a state. Its groups are $\{x_1, x_2\}$ and $\{x_3\}$, with $\min\{A,B\} = \frac{8}{27}$ on the first group and 0 on the second. So $K_\rho(M) = \frac{8}{27} \approx 0.30$, and a memory with h_1 and h_2 in different states has $K_\rho = 0$. Merging the two histories adds $\frac{8}{27}$ of irreducible, normalised regret. It is irreducible in the sense that it cannot be eliminated by a choice of policy.*

Corollary 3.5 (Regret due to memory and decisions) *For each group, I choose $q_C^\star \in \arg\min_q \{A_C(1-q) + B_C q\}$, a probability that minimises that group's loss. Every policy that uses only M satisfies*

$$\sum_i \mu_i \operatorname{loss}_i(\pi) = K_\ell(M) + \sum_{C \in \mathcal{C}_M} |A_C - B_C|\,|q_C - q_C^\star|.$$

Proof If $A_C > B_C$, then $q_C^\star = 1$ and $A_C(1-q_C) + B_C q_C = B_C + (A_C - B_C)(1-q_C)$. Here B_C is the least loss in group C, and the remaining term is the extra loss from

using q_C instead of $q^\star_C$. The case $A_C < B_C$ is symmetric. In a tied group, the loss is A_C whatever q_C, so the extra loss is zero. The sum over groups gives the stated decomposition. ■

Interpretation. Total regret is the sum of two parts. One part is due to the structure of memory, and the other part is due to execution. The latter can be eliminated by the right choice of policy, while the former cannot. Like the bias-variance decomposition in supervised learning, this split shows which part of the regret is structural and which part is fixable. It is only by changing how memory groups histories that I might reduce this structural part. Suppose an agent may use one of several memories. By Theorem 3.3, the least regret of a policy that uses only M is $K_\ell(M)$. So the agent does best (minimises regret) with the memory that has the smallest K_ℓ. Suppose one of these memories maps every history on the support to a different memory state. Then no two histories share a memory state, so that memory has $K_\ell = 0$. The agent's least regret is then zero, because it never has to group histories together and lose information. A search over memories is therefore meaningful only within a set restricted by memory size, architecture, response time or some other bodily limit. After all, the complexity of an agent's history is a subjective (Leike and Hutter, 2015) product of limited space (Bennett, 2024a), because a bounded system can only contain a finite amount of information (Bekenstein, 1981). In Section A I extend the reduction to predictions with more than two possible values, give its measurable form and its stability under estimation error, and add a regret order on memories.

3.3 When regret is freedom lost

Recall the support is the list of points, each one of which is a history-task pair with a probability. I shall now restate this setup in Stack Theory, wherein each point becomes a demand, and each deterministic policy that uses only M becomes a statement. I use the special case of a *degenerate* vocabulary, one in which every set of programs can be true at the same time (Remark 2.7). Normally, Stack Theory excludes such vocabularies because they represent systems incapable of reasoning about mutually exclusive states. However, such a system is needed for the exact reduction to a quantity in RL, since RL is not normally so concerned with embodiment.

To begin, I must define the environment. Normally in Stack Theory the environment is a set of differences lacking any semantic or ontological content. For simplicity here, however, I shall define the environment as $\Phi := 2^{\mathcal{C}_M \times \{0,1\}}$, whose states are sets of pairs of a group and a prediction, as any deeper philosophical discussion is unnecessary for this paper. For each group C and prediction b, let $a_{C,b}$ be the "declarative program" *b predicted for C*, which holds in the states that contain (C, b), much like a logical proposition. Every set of these programs holds in the state $\mathcal{C}_M \times \{0, 1\}$, so their vocabulary $\mathfrak{v}_M$ is degenerate. Each deterministic policy g that uses only M has one prediction $g(C)$ for each group C. I represent that policy by the statement $\pi_g := \{a_{C,g(C)} : C \in \mathcal{C}_M\}$, which I call a *table*. The child task in this vocabulary is $\alpha := \langle\{\mathfrak{v}_M\}, \{\mathfrak{v}_M\}\rangle$. Every statement is correct for this child task, because $\mathfrak{v}_M$ is a completion of every statement. The unseen outputs of α are then all the statements other than $\mathfrak{v}_M$.

The demand of point x_i in group C is the unseen output $u_i := \mathfrak{v}_M \setminus \{a_{C,1-y_i}\}$, which contains every program except the one whose prediction on C is wrong for x_i. The point

has weight $w_i := \mu_i \ell_i$. Points in the same group with the same best prediction have the same demand, and their weights add. Take a parent task in the same vocabulary whose correct outputs are those of α together with a set S of demands. A statement is correct for that parent if and only if it rules out no demand in S, which is the survival condition of Theorem 2.5.

As in Proposition 2.6, the weighted freedom of a statement ϑ is $W(\vartheta) := \sum_{i:\, u_i \in \mathrm{Ext}(\vartheta)} w_i$. With $Z := \sum_i w_i$, the *freedom lost* by ϑ is $Z - W(\vartheta)$, which is the weight of the demands ruled out by ϑ. Let $W^\star(M)$ be the largest weighted freedom of a table.

Corollary 3.6 (Regret is freedom lost) *For every memory M and every loss weight above,*

$$\inf_{\pi \text{ uses only } M} \sum_i \mu_i \,\mathrm{loss}_i(\pi) \;=\; Z - W^\star(M) \;=\; K_\ell(M).$$

Proof A demand u_i lies in $\mathrm{Ext}(\pi_g)$ if and only if $\pi_g \subseteq u_i$. This inclusion holds if and only if g gives the best prediction y_i for the group containing x_i, because u_i excludes only the program that predicts wrongly for x_i. So the freedom lost by π_g is $\sum_C \big(A_C \mathbf{1}\{g(C) = 0\} + B_C \mathbf{1}\{g(C) = 1\}\big)$. By Lemma 3.1, this sum is also the loss of g. Its least value over g is $\sum_C \min\{A_C, B_C\}$. The claimed equalities then follow from Theorem 3.3. ■

Interpretation. A table is a deterministic policy that uses only M, written as one program for each group. The demand of a point contains every program except the one that predicts wrongly for that point. A table rules out a demand when it contains a program that the demand is lacking. So a table rules out the demand of a point when, and only when, its prediction for that point's group is wrong. A table discards the points it predicts wrongly. The freedom lost by a table is the weight of the demands it rules out, so it is the sum of the weights of the discarded points. By Lemma 3.1, the table's loss is the same sum, so the freedom lost by a table equals the loss ascribed to that table. Every set of programs in a degenerate vocabulary can be true at the same time, because the system has no ability to represent mutual exclusivity. For this reason, a demand can omit one program while containing every other program. Therefore freedom lost by a statement is the total weight of the points for which that statement includes a program that predicts wrongly[4]. Every table contains one program per group, so by Remark 2.7 all tables have the same unweighted freedom $2^{|\mathcal{C}_M|}$, and tables differ only in weighted freedom. A policy that uses only M gives one prediction to every history in a group, so the least weight that must be deleted from the support is the least freedom lost by a table. A statement may contain programs for both predictions in a group, but it then rules out every demand of that group, so no statement with a prediction for every group is freer than the best table. By Proposition 2.6, generalisation depends on how much of the unseen future remains available to a policy, weighted by how likely the world is to impose each demand. So when histories share a memory state, regret is the lost part of the unseen future **when the vocabulary is degenerate**.

4. Note that freedom lost and regret can diverge in non-degenerate vocabularies.

Remark 3.7 (The margin weight is the prior weight) *With $\ell = \rho$, the weight $w_i = \mu_i\rho_i$ is the margin weight on wrong predictions in the normalised regret of the selection theorems of Nayebi (2026). Under the independent prior of Proposition 2.6, the same weight is the Stack-Theoretic prior weight $-\log(1-r_i)$ when $r_i = 1-e^{-\mu_i\rho_i}$ (Bennett, 2025). Under that prior, the best policy that uses only M survives with probability $e^{-K_\rho(M)}$ (Section B). Nayebi reached this weight from regret, and in Stack Theory it follows from the probability of generalising.*

When the prediction for each group can be chosen independently, the least freedom lost is the Bayes risk.

4 Decision rules, new tasks and actions

A deployed agent seldom stores its predictions in a table with one entry per memory state and task. A recurrent network instead passes its hidden state through a linear layer, so the predictions for all histories depend on a single weight vector. What does an agent lose when its predictions come from such a layer, when deployment involves new tasks, or when its actions shape what it observes? In each case, regret is at least freedom lost. For predictions from a layer and for new tasks, the least regret equals that freedom lost.

4.1 Choosing predictions from memory

Definition 4.1 (Decision rules and the freedom lost with them) *A* decision rule *for a memory M is a function g that assigns a prediction to each memory state and task, $g(M(h), T) \in \{0, 1\}$. Here G is a nonempty set of decision rules, such as the rules computed by linear layers on the memory state. The policy $g \circ M$ predicts $g(M(h), T)$ for history h and task T. A policy* uses M through G *when it is $g \circ M$ for some $g \in G$, or a random mixture of such policies. The* freedom lost with G *on M is*

$$D_G(M) := \min_{g \in G} \sum_i \mu_i \ell_i \, \mathbf{1}\{g(M(h_i), T_i) \neq y_i\}.$$

The set G_{tab} of lookup tables *contains every function that assigns a prediction to each group.*

A decision rule gives one prediction for each group. So in the degenerate vocabulary $\mathfrak{v}_M$ of Section 3, the policy $g \circ M$ is the table π_g. By the proof of Corollary 3.6, the freedom lost by π_g equals the loss of $g \circ M$. Let $W^\star_G(M)$ be the largest weighted freedom of a table π_g with $g \in G$. Then $D_G(M) = Z - W^\star_G(M)$.

Theorem 4.2 (The least regret is the freedom lost) *For every memory M, set of decision rules G and loss weight,*

$$\inf_{\pi \text{ uses } M \text{ through } G} \sum_i \mu_i \operatorname{loss}_i(\pi) \;=\; D_G(M) \;=\; Z - W^\star_G(M) \;\geq\; K_\ell(M).$$

The final inequality is an equality if and only if some rule in G gives the prediction with greater weight for every group where $A_C \neq B_C$. With G_{tab}, the freedom lost is $K_\ell(M)$.

The proof is in Section A.

Interpretation. Bayes risk is computed separately for each group, so the prediction with the greater weight can be chosen in every group. With a linear decision rule, one weight vector applies to every group, so the predictions for different groups are linked. Under the best rule, some group may then receive the prediction with the smaller weight. Example B.2 in Section B contains such a memory, with $K_\rho = 0.3$. For that memory, the freedom lost with constant decision rules is 0.4. So the least regret of an agent depends on its decision rules as well as its memory. When the rules link the predictions for different groups, Bayes risk can understate the least regret, while the freedom lost with the rules equals that least regret.

Remark 4.3 (Continuous memories) *Two histories seldom share the hidden state of a recurrent network, because that state is a vector of floating-point numbers. So almost every group contains a single point. A group with a single point loses nothing, because a lookup table can give that point its best prediction. The freedom lost with lookup tables is therefore at or near zero, and by Theorem 4.2 that freedom lost is $K_\ell(M)$. In such a memory, freedom is lost when the decision rule gives the same prediction to two histories whose best predictions differ. With linear rules, some freedom is lost whenever no linear rule gives the best prediction at every point with positive weight. Now take a recurrent network whose predictions come from a linear layer. Its least regret is the freedom lost with linear decision rules, and its K_ℓ is at or near zero. The same is true of every other recurrent network under consideration. So K_ℓ is near zero for all of these networks, while the freedom lost with linear rules can differ from one network to another.*

4.2 New tasks

During deployment, an agent may face tasks on which it was never trained. Possible deployment tasks form a finite set $\mathcal{T}$. A prior Q assigns a probability to each task in $\mathcal{T}$. Each history h_i has probability μ_i. For each task $T \in \mathcal{T}$, the history has a best prediction $y_{i,T}$ and a loss weight $\ell_{i,T}$. At deployment, a task T is drawn from Q, and the agent learns which task was drawn. Only then is a decision rule for T fitted. The rule takes the memory state as input and comes from G_T, the set of rules allowed for T. For each task T, let $D_{G_T,T}(M)$ be the freedom lost with G_T on the support $\{(h_i, T)\}_i$, as in Definition 4.1. The *freedom lost over tasks* is the average of these under Q,

$$D_Q(M) := \sum_{T \in \mathcal{T}} Q(T)\, D_{G_T,T}(M).$$

Proposition 4.4 (Unknown tasks) *1. The freedom lost over tasks is the least expected loss of a policy that uses M through G_T once T is named.*

2. On the support of all pairs (h_i, T), I group points by memory state and task. I build a degenerate vocabulary from these groups as in Section 3. Each pair has its demand $u_{i,T}$ in that vocabulary, with weight $Q(T)\mu_i\ell_{i,T}$. For each task T, I choose one rule $g_T \in G_T$. The statement ϑ contains the prediction of g_T for each group with task T. The least freedom lost by such a statement is $D_Q(M)$. If the world imposes the

demand of each pair independently with probability $1 - e^{-Q(T)\mu_i \ell_{i,T}}$*, the policy that uses* M *through the best rule for each task survives with probability* $e^{-D_Q(M)}$*. Minimising* D_Q *over memories maximises that survival probability.*

3. *Regret on a known task* T_0 *is the case where* Q *puts all its probability on* T_0*. Two memories can be ranked one way by regret on* T_0 *and the opposite way by the freedom lost over tasks.*

The proof is in Section A.

Interpretation. Take two histories with the same best predictions for the known tasks. If they share a memory state, regret on the known tasks is unchanged, whatever their best predictions for other tasks. Under a uniform prior, every task in $\mathcal{T}$ has equal weight. So the freedom lost over tasks grows with every pair of histories that share a memory state and differ in their best prediction for some task. Suppose also that every loss weight is one and every history is equally likely. Then the freedom lost over tasks is unweighted. That unweighted case is the setting of the Sufficiency theorem (Bennett, 2025), applied to the demands of every task. That theorem states that the correct policy with the most freedom is the most likely to generalise under a uniform prior over demands. The memory with the least freedom lost over tasks is then the memory whose states allow the most correct predictions across all tasks. Consider two memories with the same regret on the known task. Under one memory, the states allow correct predictions on new tasks. Under the other, histories that those tasks distinguish share a state. Choosing by regret on the known task can then deploy the memory under which those histories share a state.

A *selector* assigns a score to each trained agent before deployment, and the agent with the best score is deployed. The trained agents in this choice are the *candidates*. *Freedom over all tasks* is the selector that deploys the candidate whose memory has the least D_Q. The prior Q depends on the experiment. In the delayed-cue experiment of Section 7, each episode has one of four hidden classes. Each task there splits the four classes into two nonempty sets, and the agent must predict which set contains the hidden class. A split and its reversal are one task, so there are seven tasks, and the prior gives each of them the same weight. On Repeat Previous, the prior gives each lag a weight proportional to the number of steps in an episode at which that lag can be evaluated.

4.3 When actions change what the agent observes

The agent now acts in a partially observable Markov decision process lasting a finite number H of steps, and its actions shape its history. The value $Q_t^\star(h, a)$ is the expected return from step t onward when the agent takes action a after history h at that step and then acts as well as any policy that depends on the whole history. The best value over actions is $V_t^\star(h) := \max_a Q_t^\star(h, a)$. At each step t, the agent applies a decision rule $g_t \in G_t$ to its memory state $M_t(h_t)$ and takes the resulting action. It may also mix such rules at random. When agent π acts in the environment, its histories h_t have distribution d_t^π. After history h_t, the agent takes action a with probability $\pi_t(a \mid h_t)$. Its expected return from the initial history h_0 is $V_0^\pi(h_0)$.

Theorem 4.5 (Regret on the agent's own episodes) *For every such agent π,*

$$V_0^\star(h_0) - V_0^\pi(h_0) \;=\; \sum_{t=0}^{H-1} \mathbb{E}_{h_t \sim d_t^\pi}\Big[V_t^\star(h_t) - \textstyle\sum_a \pi_t(a \mid h_t)\, Q_t^\star(h_t, a)\Big] \;\geq\; \sum_{t=0}^{H-1} D_t(\pi),$$

where $D_t(\pi) := \min_{g \in G_t} \mathbb{E}_{h_t \sim d_t^\pi}\big[V_t^\star(h_t) - Q_t^\star(h_t, g(M_t(h_t)))\big]$. Every rule in this minimum is evaluated under the same history distribution, d_t^π. If the agent chooses between two actions, 0 and 1, at step t, then $D_t(\pi)$ is the freedom lost with G_t on the support d_t^π. For these two actions, the best prediction at h is the best action $\arg\max_a Q_t^\star(h, a)$, and the loss weight is $|Q_t^\star(h, 1) - Q_t^\star(h, 0)|$.

The proof is in Section A.

Interpretation. An agent's actions determine which histories it meets, so the bound for each agent holds on that agent's own histories. An agent that walks somewhere else is judged on other histories. To estimate the bound for a candidate, I run the candidate in the deployment environment and record its histories, without rewards. So a selector can rank candidates by the bound before any reward from deployment exists.

4.4 Three ways to lose freedom

Regret so far is measured against the best policy that depends on the whole history. But did the history hold enough information to predict the outcome correctly? Let $z \in \{0, 1\}$ be the hidden variable that the agent must predict, and compare the agent with a benchmark agent that observes z. Let $p(h) := \Pr(z = 1 \mid h)$ be the probability that $z = 1$ after history h. A wrong prediction after h loses a stake $s(h) \geq 0$. Let $y(h) = \mathbf{1}\{p(h) \geq \frac{1}{2}\}$ be the best prediction after h, and let $a(h)$ be the prediction of an agent that uses M through G.

Proposition 4.6 (Three-term split) *The agent's loss against the benchmark agent splits as*

$$\mathbb{E}\big[s\,\mathbf{1}\{a \neq z\}\big] = \underbrace{\mathbb{E}\big[s \min\{p, 1-p\}\big]}_{\textit{never gained}} + \underbrace{D_G^s(M)}_{\textit{lost in memory}} + \underbrace{\mathbb{E}\big[s|2p-1|\mathbf{1}\{a \neq y\}\big] - D_G^s(M)}_{\textit{left unused}},$$

where $D_G^s(M) := \min_{g \in G} \mathbb{E}[s|2p-1|\mathbf{1}\{g(M(h)) \neq y(h)\}]$. Every term is nonnegative.

The proof is in Section A.

Interpretation. Freedom never gained is the loss that the best policy on the whole history still incurs, because the history does not determine z. It is a lacuna in the history itself, and it can be reduced only by acting differently, for example by further sensing. Freedom lost in memory is the least regret of Theorem 4.2. It arises when histories with different best predictions share a memory state. Freedom left unused is how much more the agent loses than the best decision rule in G for its memory. A selector that scores only freedom lost in memory chooses an incurious agent, one that remembers everything it observes and observes almost nothing. So when agents differ in how much they sense, a selector must also score the freedom never gained.

4.5 Imperfect teachers

Wherever p is unknown, I compute each selector's score with a teacher's estimate $\hat{p}$ in place of p. The teacher is a model trained on labelled source episodes, and I describe it together with the selectors in Section 5. At point i, prediction a has loss $L_i(a) := \ell_i \mathbf{1}\{a \neq y_i\}$. The estimate $\hat{L}_i(a)$ is the same loss computed with $\hat{p}_i$ in place of p_i. The freedom lost computed with $\hat{L}$ in place of L is $\hat{D}_G(M)$. At each point, the largest error in estimated loss across the possible predictions is $\varepsilon_i := \max_a |\hat{L}_i(a) - L_i(a)|$. The teacher's error is $\varepsilon := \sum_i \mu_i \varepsilon_i$, the average of these errors with weights μ_i.

Proposition 4.7 (Teacher error) *For every set of decision rules G and memory M, $|\hat{D}_G(M) - D_G(M)| \leq \varepsilon$. If $\hat{M}$ minimises $\hat{D}_G$ over a set $\mathcal{M}$, then $D_G(\hat{M}) \leq \min_{M \in \mathcal{M}} D_G(M) + 2\varepsilon$. For a binary prediction task, $\varepsilon_i \leq 4|\hat{p}_i - p_i|$ for normalised regret, $\varepsilon_i \leq 2|\hat{p}_i - p_i|$ for success and $\varepsilon_i \leq 2s_i|\hat{p}_i - p_i|$ for return.*

The proof is in Section A.

Interpretation. Suppose I choose a memory from a set $\mathcal{M}$ by the teacher's estimates. The *selection loss* is the freedom lost with the chosen memory minus the freedom lost with the best memory in $\mathcal{M}$. With a teacher whose error is ε, the selection loss is at most 2ε. The bound holds for every set of memories and every environment. So a teacher with a small error suffices to choose a memory almost as well as if p were known.

5 Estimating freedom without rewards

Every quantity in Section 4 depends on p, and p is unknown in the target environment until rewards exist. If p is estimated without target rewards, each quantity can serve as the score of a selector.

Teacher. Source episodes come with labels or rewards. So after each source episode, the hidden variable is known, and with it the correct prediction. A teacher is a model trained on those labelled source episodes. It maps a history to a probability for each value of the hidden variable. The estimate $\hat{p}_T(h)$ for every task T follows from these probabilities. Here the teacher is never deployed, and its only job is to label histories. In experiments E1 to E3 (Section 7), the primary teacher is a set network. It converts each step of a history into a vector, combines those vectors into one, and maps that vector to a probability for each value of the hidden variable. Combining the vectors discards the time of each step. So for a history with a longer delay, the combined vector differs only by the vectors of the extra steps without a cue.

Unlabelled target sample. Each selector has access to a sample of unlabelled target histories without rewards, the *target sample*. In environments where the agent's predictions do not change observations, every candidate shares one target sample. Where actions change observations, I run each candidate in its own target episodes, because the bound of Theorem 4.5 holds on the candidate's own histories. The teacher labels every history in the target sample with a best prediction $\hat{y}$ and a weight $\hat{\ell}$. In Repeat Previous, the suit at every lag is a function of the observations, so the labels follow from the observations and no teacher is needed.

Estimators. For a memory with finitely many states, I group the target sample by memory state and task, then estimate K_ℓ with the formula in Theorem 3.3. The estimate is $\hat{K}_\Delta$ with success weights, $\hat{K}_\rho$ with normalised-regret weights and $\hat{K}_1$ with unit weights. Now take a continuous memory whose predictions come from a linear layer. For such a memory, $\hat{D}_{\text{lin}}$ is my estimate of $D_{G_{\text{lin}}}$, the freedom lost with linear decision rules. To compute it, I train a logistic decision rule on the teacher's labels, weighting each history by the teacher's weight. The estimate $\hat{D}_{\text{lin}}$ is the weighted error of that rule on histories held out from its training. For comparison, I also estimate the freedom lost with a k-nearest-neighbour decision rule. A k-nearest-neighbour rule can realise more tables than a linear rule can.

A candidate may never reach the decision point in its own episodes. For example, an agent in the T-Maze of Section 7 may stop short of the junction. Such an episode ends without a decision, so I add the share of such episodes to the freedom lost. Let reach be the share of a candidate's own target episodes in which it reaches the decision point. With success weights, the predicted success of a decision rule refitted on the target sample is then $\text{reach} \times (1 - \hat{D}_{\text{lin}})$. So in the T-Maze, the *freedom selector* deploys the candidate with the least $1 - \text{reach} + \text{reach} \cdot \hat{D}_{\text{lin}}$, which is one minus that predicted success. When agents differ in how much they sense, the freedom selector instead minimises a sum of three terms. The three terms are

1. the teacher's estimate of the freedom never gained, $\hat{\mathbb{E}}[s \min\{\hat{p}, 1 - \hat{p}\}]$,
2. the estimate of the freedom lost in memory, and
3. the return lost on episodes in which the agent never reaches the decision point.

I test these sums against selectors that score a single term. The selector *memory term only* scores the freedom lost in memory on the episodes in which the agent reaches the decision point. The selector *never-gained term only* scores the teacher's estimate of the freedom never gained.

Which selector for which deployment. Each way of deploying an agent has its own estimate of the deployed agent's loss.

1. The agent may ship unchanged. Its target return is then estimated by *teacher evaluation*. Teacher evaluation compares the candidate's own action probabilities on its target sample with the teacher's labels. So teacher evaluation estimates the regret on the left side of Theorem 4.5, freedom left unused included.
2. The memory may stay while a new decision rule is fitted on the target sample with the teacher's labels, as in last-layer retraining (Kirichenko et al., 2023). The loss of the refitted rule is at least the freedom lost with rules of that kind (Theorem 4.2), and the best such rule attains it. For this deployment, the estimate is therefore the freedom lost.
3. The tasks may be specified only after the memory is chosen. For this deployment, the estimate is the freedom lost over tasks (Proposition 4.4).
4. Candidates may differ in how much they sense. For this deployment, the estimate includes the freedom never gained (Proposition 4.6).

Four predictions follow, and the tests of all four are in Section 8.

1. Teacher evaluation should beat the other selectors when agents ship unchanged.
2. Choosing by freedom should beat them after the decision rule is refitted.
3. Freedom over all tasks should beat them on new tasks, and lose or tie on the known tasks.
4. The selector *memory term only* should fail when agents differ in what they sense.

6 Related work

Selection theorems. Three lines of work argue that competence requires internal structure, namely mine, that of Richens and colleagues, and that of Nayebi. In 2023 I proved that freedom maximisation is optimal for generalisation under maximal ignorance (Bennett, 2023a). The same year, I showed that causal identities emerge from freedom maximisation under stated preconditions, where a causal identity is a representation of an intervention (Bennett, 2023b). I collected and extended both results in my thesis (Bennett, 2025). Richens and Everitt (2024) then derived in RL terms that robust agents must learn approximate causal models. Richens et al. (2025) derived that agents generalising to multi-step goals must encode a predictive world model. The causal-identity result therefore predates that of Richens and Everitt (2024). The two are approximately one claim, derived in different formalisms. On the causal side, this work builds on structural causal models (Pearl, 2009) and the causal hierarchy (Bareinboim et al., 2022). Nayebi (2026) works in a POMDP betting setting and derives quantitative selection theorems for predictive state, memory, modularity, regime tracking and recoding. These theorems have no Stack-Theoretic antecedent. By Theorem 3.3 and Corollary 3.6, Nayebi's least normalised regret is the freedom lost in a degenerate vocabulary. In Section 4 I extend that identity to decision rules, new tasks and actions.

Representations for unknown rewards. Successor features transfer to rewards expressed as weighted sums of features fixed in advance (Barreto et al., 2017). The forward-backward method learns one representation from which a policy for each reward can be derived (Touati and Ollivier, 2021). In Proposition 4.4, a memory is scored against a prior over tasks, and the prior is uniform when the tasks are unknown. Training on additional tasks changes what a memory holds, beyond what the training reward alone would require (Jaderberg et al., 2017). I use such tasks to vary the candidates in the part of the delayed-cue experiment where agents face new prediction tasks at deployment (Section 7).

Model selection under changed conditions. Many methods choose among trained models without target labels, for target conditions that differ from those of training. Importance-weighted validation weights each source loss by the ratio of target probability density to source probability density (Sugiyama et al., 2007). Deep embedded validation estimates this ratio from learned features. It reduces the variance of its loss estimate with a control variate, a correction proportional to the mean weight minus one (You et al., 2019). Label-free methods rank models by their confidence on target inputs, as with maximum softmax probability (Hendrycks and Gimpel, 2017), entropy (Morerio et al., 2018), average

thresholded confidence (Garg et al., 2022) or disagreement between models (Jiang et al., 2022). All of these concern how a model behaves on target inputs, and none concerns which histories share a state in its memory. In the T-Maze results of Section 8, the agents chosen with these scores are therefore cocksure of the wrong turn.

Off-policy evaluation and policy selection. In offline RL, policies are selected from recorded interactions without collecting new rewards (Paine et al., 2020). Off-policy estimators are scored on benchmarks by the rank correlation between estimated and realised values over a set of policies (Fu et al., 2021). That rank correlation is the statistic I use for the selectors of experiment E5 (Section 7). Those estimators require recorded rewards from the target environment, while the selectors here require no target rewards and at most a teacher trained on source labels. Teacher evaluation estimates the return of an unchanged agent directly, so I include it as the strongest rival to choosing by freedom.

State abstraction and bisimulation. In state abstraction, the states of a Markov decision process are grouped, and Li et al. (2006) analyse which groupings preserve optimal planning. Bisimulation groups states by their transitions and rewards, and it is the finest grouping of practical interest (Givan et al., 2003). Bisimulation metrics bound the value lost by grouping (Ferns et al., 2004, 2011), and deep RL methods learn representations that respect these metrics (Zhang et al., 2021; Castro, 2020; Gelada et al., 2019). In Section 3, by contrast, history-task pairs are grouped when they share a memory state and a task. Theorem 3.3 states the exact regret of that grouping. Once the predictions for different groups depend on one decision rule, the grouping determines only a lower bound on the regret (Theorem 4.2).

Probing and usable information. A linear probe is a linear classifier fitted to fixed features, to measure what those features encode (Alain and Bengio, 2017). In RL, such probes are used to test which state variables a learned representation holds (Anand et al., 2019). The freedom lost with a linear layer (Section 5) is a probe of this kind, with each point weighted by its loss weight. How flexible a probe should be is debated (Hewitt and Liang, 2019; Pimentel et al., 2020). For selection, the answer follows from Theorem 4.2. The least regret with a set of decision rules is the freedom lost with those rules, so the probe should use the kind of decision rule that will be deployed. The most flexible probe is a lookup table, and under it every recurrent candidate scores at or near zero (Remark 4.3). Xu et al. (2020) measure usable information by the least log loss attainable within a restricted set of predictors. The freedom lost with a set of decision rules is the least loss within that set under weighted 0-1 loss instead of log loss. The selector *teacher log loss* scores the least log loss among lookup tables, computed against the teacher's probabilities (Section 7). On new tasks, choosing by teacher log loss closed 37% of the gap between random choice and the best candidate. Freedom over all tasks closed 91% of that gap (Table 5).

Memory in RL. Tuned recurrent model-free RL matches specialised POMDP methods (Ni et al., 2022), and transformers extend usable memory (Ni et al., 2023; Parisotto et al., 2020). The POPGym (Morad et al., 2023) and POBAX (Tao et al., 2025) suites contain memory tasks, including the T-Maze of Bakker (2001). Here I study a different problem, which of several trained memories to deploy.

Teachers and last-layer retraining. Training with information that the deployed agent lacks is standard, from asymmetric actor-critics (Pinto et al., 2018) to learning by cheating (Chen et al., 2020) and distillation (Hinton et al., 2015). Retraining only the last layer on a small labelled set reduces a network's reliance on spurious features (Kirichenko et al., 2023). Here, last-layer retraining uses teacher labels in place of target labels. Theorem 4.2 states the least regret attainable by refitting.

7 Experimental design

Given a pool of trained agents and unlabelled histories from a new environment, which agent should be deployed? I study this problem in four environments, whose full settings are in Section C. Every proposed selector deploys the candidate with the least freedom lost, for a named set of decision rules and a named prior over tasks. So each comparison tests a claim of Section 4 inside RL.

7.1 Protocol

A *replication* is one independent run of an environment, with its own training seeds, its own pool of agents and its own episodes. The replication is the unit of analysis, and every comparison is paired within a replication. Each replication runs in two stages, so that each selector's choice precedes every target outcome. In the *selection stage*, I train the pool on source episodes, fit the teachers on labelled source episodes and collect the target sample. I then compute every selector's scores and every refitted decision rule, and save them in a *selection record*. Only after saving every replication's selection record do I begin the *evaluation stage*. In the evaluation stage, I generate fresh target episodes that no selector has used and measure what each candidate achieves. No replication was included, excluded or rerun because of a selector's behaviour or a target outcome. For each experiment, I first ran a pilot and then fixed a plan before the confirmatory run, which used none of the pilot's random seeds. I describe the pilots and the two amendments in Section C. Neither amendment followed a target outcome.

7.2 Environments

E1, delayed cue. Each episode has one of four equally likely hidden classes. The agent observes noisy cues of the class at steps 1 to 3, and then a delay filled with distractor symbols. Three prediction steps follow, each requiring a yes-or-no prediction about the class. Together the correct predictions identify the class. A correct prediction earns reward one. Predictions never change later observations, so memory is isolated from exploration. Each episode is in one of two regimes, and the agent observes the regime. The regimes differ in how reliable the cues are. So the loss weight of a wrong prediction varies with the regime and with how well the cues agree. The rare regime is 5% of source episodes. In target episodes its share is 30%. E1 has two parts. In the part with longer delays, the teachers and source validation use source episodes with delays of 3 to 7 steps. Target delays are 40, 60 or 80 steps, so the tasks are unchanged and the memory must hold the cue through a longer delay. In the part with new tasks, source and target delays are both 3 to 7 steps, and the change is in the tasks. Agents in this part train on one task at every prediction step and are scored on

that task. At deployment, the agent performs all three tasks, so two of them are new. The known task is the task on which the agents trained, and it serves as a control.

E2, T-Maze. E2 is the POBAX T-Maze (Bakker, 2001; Tao et al., 2025), re-implemented in NumPy with the POBAX dynamics and rewards. The goal side is visible only at the starting position. The agent walks along a hallway and turns north or south at the junction. The correct turn earns 4. A wrong turn loses 0.1. Here movement determines what the agent observes, and an agent that stops walking never reaches the junction. Source hallways are 5 to 10 grid squares long. Target hallways are 20, 40 and 80 grid squares long in equal shares, so the task is unchanged and the agent must walk further before it can use what it remembers.

E3, noisy T-Maze. For E3 I stipulated RockSample (Smith and Simmons, 2004) and a fallback in the plan fixed in advance. The fallback is a T-Maze in which the agent decides how often to sample a noisy cue. E3 ran as the fallback after a contretemps in the RockSample pilot, where every agent learned to leave at once, sensing nothing (Section C). At each step at the starting position, the agent observes a fresh noisy sample of the goal. Moving north or south there does nothing, so staying to sample again is the sensing action. Each episode has a visible regime, either ordinary or risky. The regime determines how reliable the samples are and how much a turn is worth. In the risky regime the samples are less reliable. A turn in that regime is worth more. Source hallways are 5 to 10 grid squares long, and target hallways are twice as long. The risky regime is 10% of source episodes. In target episodes its share is half. Agents here differ in how long they sample the cue, what they remember and how they act. These three differences correspond to the three terms of Proposition 4.6.

E4 and E5, Repeat Previous. In E4 and E5 I run POPGym's Repeat Previous (Morad et al., 2023) from the published package. A standard deck of 52 cards is shuffled, and at each step the agent observes the suit of the next card. An episode lasts 51 steps. At step $t \geq k$, the task with *lag* k is to predict the suit of the kth most recent card, so lag 1 concerns the current card. A correct prediction earns $1/(52-k)$ and a wrong one loses as much, so a perfect agent scores one. Predictions never change later observations. Agents train on lag 4. At deployment, they are evaluated on the new lags 2, 3 and 5 to 8, and on lag 4 as a control. E5 is a rerun of E4 on fresh seeds, with a new confirmatory statistic.

7.3 Pools, teachers and selectors

Every agent trains with recurrent proximal policy optimisation (PPO) (Schulman et al., 2017) on episodes from the source environment. I add two saved versions of each agent to the pool, one saved halfway through training and one at its end. The delays and regime shares of these training episodes may differ from those of the source episodes used by the teachers and by source validation. A gated recurrent unit (GRU) or long short-term memory (LSTM) layer updates its memory after each observation. In E1, the pool includes every combination of recurrent layer type, width, range of training delays and share of the rare regime in training (Section C). The type is GRU or LSTM, and the width is the number of units in the recurrent layer. In the part with new tasks, candidates vary in whether they have an auxiliary head, instead of in their range of training delays. An auxiliary head is an additional predictor trained to identify the most recent cue. I round each memory to six

states with k-means, which groups memory vectors around six fitted centres. The memory's predictions can then be stored as a table with one entry per state. I round each memory in five ways, which divide the six states differently between the two regimes. In E2 to E5 the pool comprises GRU and LSTM networks of three widths and two-block gated transformers (Parisotto et al., 2020). The attention window of each transformer is 8 or 32 steps. In E4 and E5 the pool has one more architecture, a gated transformer with one block and an attention window of 4 steps. This *four-card transformer* attends only to the four most recent cards. I include it to test selectors that score only lag 4, since its memory holds little but the card at that lag. I run every confirmatory test both with and without this transformer. Before the selectors run, I compare the candidates' own policies on source episodes. I remove every candidate that falls more than a fixed margin below the best candidate in its replication. This removal depends only on source data. The agents that failed to learn are culled this way. Failed agents are easy to reject, so without them every comparison is harder for the proposed selectors. From here on, the pool means the candidates that remain. Random choice, the best agent in hindsight and every selector are restricted to that pool.

Every teacher is trained on labelled source episodes and has no access to target labels. The primary teacher is a set network over the episode's observations. In E1 I add two more teachers, a GRU and the *analytic posterior*. The analytic posterior uses the process that generates the episodes to compute the probability of each hidden class after a history. Its cue reliabilities are estimated on source episodes. To test Proposition 4.7, I add noise to the analytic posterior. In E4 and E5 no teacher is needed, because the correct suit at every lag is determined by the observations.

The proposed selectors are the freedom estimates of Section 5. Every other selector is either a rival or a single term of a proposed selector (Table 3). For *regret on the trained task*, I score each memory only on the task used for training. The score estimates the least regret attainable with the decision rules used in deployment. I compute these estimates from target histories and choose a memory with the lowest estimate. Source validation is the incumbent. It ranks agents by the performance of their own policies on source episodes excluded from training. Teacher evaluation compares each agent's own action probabilities on its target sample with the teacher's labels. Teacher log loss averages the teacher's probabilities for the hidden class within each memory state, and it scores a memory by the log loss of these averages. The remaining rivals are deep embedded validation (You et al., 2019) and four label-free scores from the model-selection literature. These four are the *confidence scores*, which measure how confident an agent looks. They are maximum action probability (Hendrycks and Gimpel, 2017), action entropy (Morerio et al., 2018), thresholded confidence (Garg et al., 2022) and disagreement with the pool (Jiang et al., 2022). In E4 and E5, the confidence scores are computed from the agent's predictions on lag 4. E4 and E5 have one more rival, network size, which deploys the network with the most weights. Freedom on source histories is the freedom lost computed on source histories instead of the target sample. The difference between its results and those of the freedom selector measures the value of the target sample. Online validation runs each candidate on n rewarded target episodes in the evaluation stage and deploys the candidate that performs best on those episodes. I measure each selector's value as the number n of rewarded episodes per candidate that online validation needs to match that selector's performance. Random choice is the average over the pool.

7.4 What I measure

The outcome is the target performance of the deployed agent, averaged over fresh target episodes. Performance is return in E3 and success elsewhere. It is measured either with the decision rule refitted or with the agent unchanged (Sections C.5 and 5). For each selector, I report the *share of the gap closed*. The gap is the difference in outcome between the best agent in hindsight and random choice. The share is a ratio of sums over replications, so that 100% matches the best agent in hindsight and 0% matches random choice. Where deployment includes new tasks, I report the normalised regret of the deployed agent on those tasks. In E5 I instead score each selector by how well it ranks the whole pool. This criterion is used to judge estimators in benchmarks of off-policy evaluation (Fu et al., 2021) and complexity measures in studies of generalisation (Jiang et al., 2020). I orient each selector's score so that higher is better. In each replication, τ is Kendall's rank correlation τ_b between that score and the candidates' success on the new lags after refitting. In E4, freedom could not be distinguished from several rivals by comparing one deployed agent per replication (Section C.8). I chose τ after that result and fixed it before the first E5 replication. Intervals are bootstrap intervals over replications. Paired comparisons are one-sided sign-flip tests on the differences within replications. Within each experiment, the p values of the group of confirmatory tests are Holm-adjusted. Several candidates may share a selector's best score. The selector then deploys each of them with equal probability, and every outcome and paired difference is the expectation under that choice.

8 Results

Each selector winnowed its replication's pool to one agent, and every such choice preceded the first target reward. Figure 1 shows the share of the gap that each selector closed in every experiment. Table 1 lists the normalised regret of the deployed agent on new tasks.

8.1 New tasks

What happens when an agent is deployed on a task on which it never trained? In E4, on POPGym's Repeat Previous, the agent observes the suit of one card per step and must name the suit of the card at lag k, the kth most recent card. Every agent trained on lag 4 and was deployed on lags 2, 3 and 5 to 8. Regret on lag 4 is the freedom lost when the prior over tasks has all its weight on lag 4. Freedom over all tasks weights every lag by the number of steps at which that lag can be evaluated in an episode. On the new lags, the agents deployed by freedom over all tasks had 69% less normalised regret than those deployed by regret on lag 4 (Table 1). Freedom over all tasks beat regret on lag 4 in every replication (Table 10). On lag 4 itself, the order reversed, and regret on lag 4 won by 0.003 in success, a Pyrrhic victory.

Figure 2 shows why. Every candidate in the pool had success 0.96 or better at lag 4 after refitting, so the candidates nearly tied under regret on lag 4. Among these near ties, regret on lag 4 settled on the four-card transformer in 50% of replications. That transformer attends only to the four most recent cards. Once trained on lag 4, it hoarded the card at lag 4 and held little of the cards at lags 2 and 3. Its predictions at those lags were therefore poor, and at every lag beyond 4 they were about as accurate as guessing. Regret on lag 4

Table 1: Normalised regret of each selector's deployed agent on the new tasks, averaged over replications. The experiments are E1 with new tasks (two new tasks, 48 replications) and Repeat Previous (six new lags, 32 replications in each of E4 and E5). Regret on the trained task means regret on the known task with success weights in E1, and regret on lag 4 in E4 and E5. On Repeat Previous, a perfect agent scores one and a wrong prediction loses what a correct one earns. Normalised regret computed from this return is twice the share of wrong predictions. The last two rows contain the reduction in normalised regret when freedom over all tasks replaces the named selector. Each reduction is a ratio of sums over replications, with a 95% bootstrap interval over replications.

Selector	E1 new tasks	E4 new lags	E5 new lags
Freedom over all tasks	0.163	0.235	0.237
Regret on the trained task	0.388	0.762	0.771
Source validation	0.375	0.269	0.247
Random choice	0.325	0.841	0.851
Best agent in hindsight	0.143	0.220	0.223
Reduction against regret on the trained task	58% [56%, 60%]	69% [68%, 70%]	69% [68%, 70%]
Reduction against random choice	50% [47%, 52%]	72% [71%, 73%]	72% [71%, 73%]

is unchanged when a memory lacks the cards at other lags. Freedom over all tasks never picked the four-card transformer, and the agents that it deployed scored at least 0.99 on every lag from 1 to 5. Without the four-card transformers in the pool, freedom still beat regret on lag 4 on the new lags in 28 of 32 replications.

In E4, I tested a second hypothesis by comparing the agent deployed by freedom with the agents deployed by seven rivals. Freedom and several rivals chose the largest recurrent network, so freedom was indistinguishable from those rivals (Section C.8). In E5, I therefore reran E4 on 32 fresh replications. I scored each selector by how well its ranking of the whole pool agreed with the candidates' success on the new lags. The measure of agreement was the rank correlation τ of Section 7, fixed before the first E5 replication. Freedom's ranking had mean $\tau = 0.71$. The rivals' mean τ ranged from 0.20 to 0.33. Regret on lag 4 had mean $\tau = 0.09$. With the whole pool, freedom's τ exceeded that of regret on lag 4 and of every rival in every replication (Table 10). When I scored one deployed agent per replication, as in E4, freedom again beat regret on lag 4 in every replication (Section C.8).

In the part of E1 with new tasks, I compare freedom over all tasks with regret on the known task, this time with a teacher in the loop. Every candidate trained and was scored on the same task at every prediction step, and deployment included that task and two new tasks. Regret on the known task is the freedom lost when the prior over tasks has all its weight on that task. The agents deployed by regret on the known task had normalised regret 0.388 on the new tasks, worse than random choice. Freedom over all tasks is computed without the list of deployed tasks, yet the agents that it deployed had a normalised regret of 0.163. That is 58% less. Freedom over all tasks fell only a little short of a reference selector

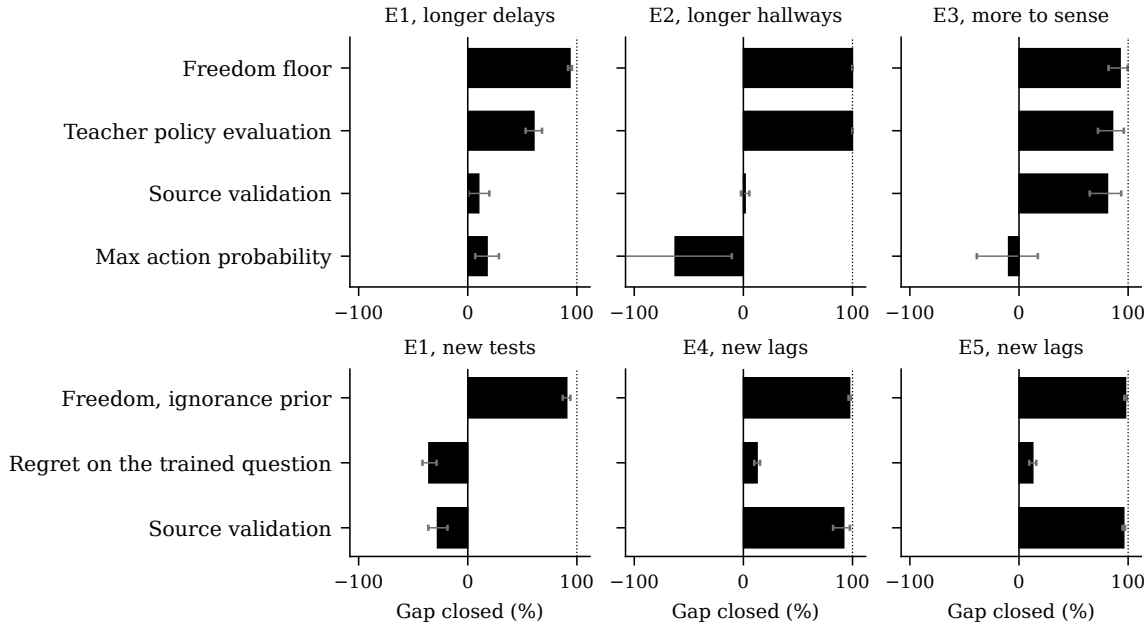


Figure 1: Share of the gap closed by each selector, after the decision rule was refitted on the target sample, with 95% bootstrap intervals over replications. The gap runs from random choice (0%) to the best agent in hindsight (100%). In the top row, deployment used the known tasks and changed how long the cue must stay in memory, how far the agent must walk or how much it must sense. There, freedom means the freedom lost with a linear decision rule in E1, and the freedom selector of Section 5 in E2 and E3. In the bottom row, deployment included new tasks. There, regret on the trained task means regret on the known task with success weights in E1, and regret on lag 4 in E4 and E5. Bars below zero mark selectors that did worse than random. Every selector, and the agents shipped unchanged, appear in the tables of Section C.6.

computed with the deployed tasks (Table 5). On the new tasks, freedom over all tasks beat regret on the known task in every replication (Table 9). On the known task the order reversed, and regret on the known task won in every replication. An agent trained on one task can perform it by storing one bit, and the classes with the same value of that bit then share a memory state. Regret on that task is unchanged by this merging, while freedom over all tasks penalises the memory's inability to distinguish those classes. Source validation uses only the known task, and it fared worse than random on the new tasks. Some candidates trained an auxiliary head that predicts the most recent cue, and freedom over all tasks beat regret on the known task even among those candidates (Section C.8).

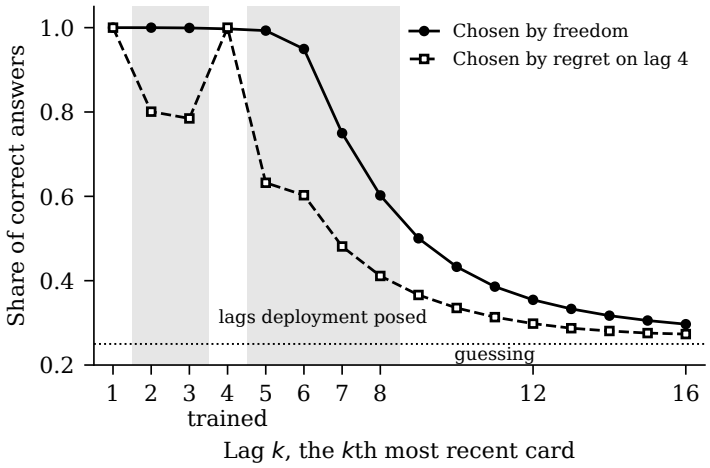


Figure 2: Repeat Previous, E4. Share of correct predictions at each lag after refitting, for the agents deployed by freedom over all tasks and for the agents deployed by regret on lag 4. Each share is averaged over the 32 replications. Every agent trained on lag 4, and the shaded lags are those evaluated at deployment. At lag k, the target is the suit of the kth most recent card, and a guess is right one time in four.

8.2 Longer delays

What if the tasks stay the same, but the memory must hold the cue for longer? In the part of E1 with longer delays, training delays were at most 28 steps. Target delays were 40 to 80 steps. Source validation ranks agents by their success on the source delays. Agents trained on the shortest delays do best there, so source validation chose those agents. For six-state memories with a table refitted on the target sample, source validation closed 19% of the gap. Choosing by $\hat{K}_\Delta$ on the target sample closed 90%. The agents' own memories are continuous hidden states. On the target sample, the freedom lost with lookup tables was zero for every one of them in every replication (Remark 4.3). With linear decision rules, the freedom lost varied between memories, and choosing by it closed 94% of the gap. Source validation closed 10% (Table 9). As predicted in Section 5, whether freedom or teacher evaluation wins depends on how the agent is deployed. Teacher evaluation did as well as freedom for agents shipped unchanged. After refitting, freedom won (Section C.8).

8.3 When movement determines what the agent observes

In the T-Maze, an agent must walk to the junction before it can use what it remembers. The freedom selector sums two terms. One is the share of a candidate's own episodes in which it never reaches the junction, and the other is the freedom lost with a linear decision rule on the remaining episodes. In every replication, this selector singled out a candidate with success 1.000. Source validation closed only 2% of the gap, because most candidates solved every source hallway and tied for the top score. Maximum action probability and action entropy did worse than random. The agents in the pool walk to the junction and

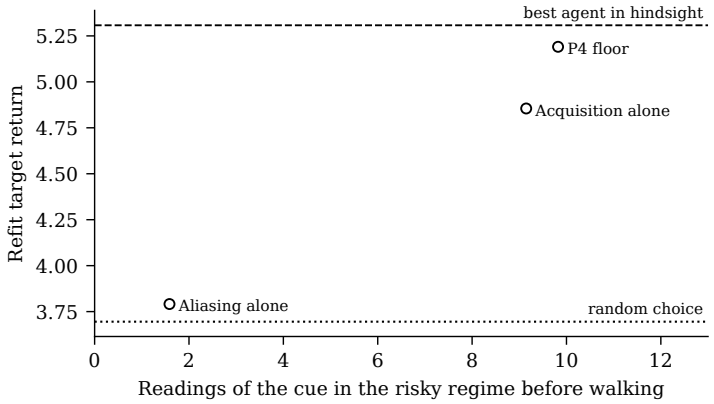


Figure 3: E3. Target return after refitting against the number of cue samples taken in the risky regime before walking, averaged over the 24 replications. The points are the agents deployed by the freedom selector, the never-gained term only and the memory term only. The dashed line marks the best agent in hindsight, and the dotted line marks random choice. Every selector appears in the tables of Section C.6.

turn with aplomb, and that confidence is unrelated to whether the cue is still in memory. Some agents remembered the cue perfectly at the junction but reached the junction in only about half of the longest hallways. So the freedom selector needs the share of episodes in which the agent never reaches the junction. Without that share, the selector *memory term only* closed 72% (Table 6).

8.4 When agents choose what to sense

In the noisy T-Maze an agent chooses how many times to sample an unreliable cue before it walks. Here the freedom selector sums three terms. They are the freedom an agent never gained by sampling too little, the freedom lost in its memory, and the return lost on episodes in which it never reaches the junction. The freedom selector closed 93% of the gap. By contrast, the selector *memory term only* closed just 6%. Figure 3 shows why. In the risky regime, the best agents in hindsight sampled the cue 10.1 times on average before walking. But the selector *memory term only* chose agents that sampled it 1.6 times, fewer even than the average agent in the pool (Table 12). An agent that samples once and walks has little to remember, and it remembers that little well. So almost no freedom is lost in its memory. Yet the agent never gained the freedom that more samples would have brought. This is Proposition 4.6 at work. A selector that ignores the freedom never gained deploys the agent that sampled the cue least. Source validation did better here than in the T-Maze, because within replications the agents that sampled more tended to score more on source episodes (Section C.8).

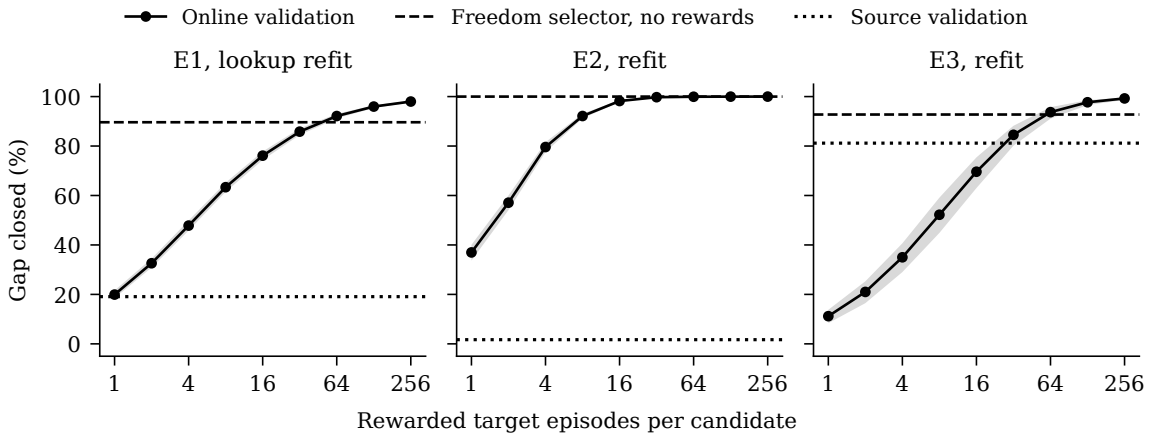


Figure 4: Share of the gap closed by online validation with n rewarded target episodes per candidate, with the decision rule refitted in each environment. The solid line with markers is online validation, and the grey band is its 95% interval. The dashed line is the freedom selector, which uses no rewards. The dotted line is source validation. The freedom selector is $\hat{K}_\Delta$ in E1 and the selectors of Section 5 in E2 and E3.

8.5 What selection without target rewards is worth

How many rewarded target episodes would it take to do as well without freedom? Online validation runs each candidate on n rewarded target episodes and deploys the candidate that performs best on those episodes (Figure 4). In the T-Maze it matched the freedom selector only at 16 to 32 rewarded episodes per candidate. For a pool of 25 candidates, that comes to several hundred rewarded target episodes. In the noisy T-Maze it needed 64 per candidate to match the freedom selector. In E1 it needed 32 to 64 per candidate to match $\hat{K}_\Delta$. With 125 six-state candidates in the pool on average, that comes to several thousand rewarded target episodes. For agents shipped unchanged, about 8 per candidate sufficed to match teacher evaluation.

9 Discussion

Where freedom and regret differ. Four results in this paper turn on the same crux. They concern new tasks, the ranking of the whole pool, continuous memories and sensing. Regret on the known tasks is myopic, since it depends only on predictions for those tasks. Freedom depends on how many further commitments remain available. When agents faced new tasks at deployment, the memories picked by freedom over all tasks had less than half the normalised regret of those picked by regret on the trained task (Table 1). In every replication of E5, freedom's ranking of the whole pool agreed with success on the new lags better than the ranking of every rival (Table 10). When the memory was a continuous hidden state, the freedom lost with lookup tables was zero for every candidate. That freedom lost is K_ℓ. The candidates differed only in the freedom lost with linear layers, which is their least

regret. When agents chose how much to sense, the selector *memory term only* deployed the agents that sampled the cue least. The freedom selector includes freedom never gained, and the agents that it deployed sampled the cue nearly as often as the best agents in hindsight. In each case, the better choice depended on the set of demands that a policy still allows, and that set is the central object of Stack Theory.

How to use freedom. A practitioner can train a pool on the source environment, fit a teacher on labelled source episodes, and collect unlabelled histories from the target environment. Where actions change observations, these histories come from each candidate's own episodes. The practitioner can then score each candidate by the freedom lost with the decision rules that will be deployed, under the prior over tasks suited to the deployment. Two terms are added to this score, the share of decisions that the candidate never reaches and the candidate's freedom never gained. Finally, the practitioner refits the chosen candidate's decision rule on the same histories with the teacher's labels. Across E1 to E3, this procedure matched online validation with 16 to 64 rewarded target episodes per candidate, and it used no rewards. An agent that must ship unchanged is better scored by teacher evaluation.

The teacher. Every proposed selector in E1 to E3 depends on a teacher trained on source labels, and the teacher must stay accurate on target histories. A set network stayed accurate on longer delays, because it discards the time of each step. With this teacher, choosing by $\hat{K}_\Delta$ closed 90% of the gap. A GRU teacher trained on short delays lost its accuracy on longer delays. With this teacher, choosing by $\hat{K}_\Delta$ closed only 68% of the gap. By Proposition 4.7, a teacher with error ε causes a selection loss of at most 2ε. In E1, the selection loss was well inside that bound (Section C.8). In E4 and E5 the suits to be predicted are functions of the observations, so no teacher is needed. A teacher trained on source episodes would fail under a shift that changes how observations bear on the hidden variable, for example cues that change meaning. That case lies outside the scope of this paper.

The effect of the weights. The identity of Corollary 3.6 has weights because regret is defined with them, but their effect on the outcomes of selection was nugatory (Section C.8). In most E1 replications, freedom chose the same memories with success, normalised-regret and unit weights. In most E3 replications, the selector *memory term only* chose the same agents with return and success weights. The weights affect the choice only when no single decision rule is best on every group. Then the weights determine which groups receive the prediction with the greater loss. In these environments, that conflict rarely arose for the best memories. Where the selections did differ, the memories chosen with success weights had more memory states for the rare regime than those chosen with unit weights. The memories chosen by source validation had the fewest memory states for the rare regime.

Scope. The environments are spartan, small enough to run 312 replications on a laptop and a cloud machine. In E2 and E3, I re-implement POBAX dynamics in NumPy rather than calling the JAX library. Each decision turns on one hidden variable or one past observation. The agent uses a linear layer or a table with one entry for each memory state and task. Where teachers are needed, they train with source labels. The theory has none of these limits, because Theorems 4.2 and 4.5 and Propositions 4.4, 4.6 and 4.7 hold for every set of decision rules, every prior over tasks, every POMDP with a finite number of steps and every teacher.

10 Conclusion

I have shown that regret is freedom lost. When two histories have the same memory state, the agent must make the same prediction for both. The least regret of an agent with that memory is the total weight of the forgone predictions (Theorem 3.3). In a degenerate vocabulary, the best table loses the same amount of freedom (Corollary 3.6). In every other vocabulary, regret and freedom lost differ in value. With a linear layer, the least regret is the freedom lost with linear rules (Theorem 4.2). On tasks on which the agent never trained, regret is silent and the freedom lost over tasks is the least expected loss (Proposition 4.4). Regret on an agent's own episodes is at least the freedom lost on those same episodes (Theorem 4.5). A teacher estimates each of these quantities from unlabelled histories (Proposition 4.7).

These results let a practitioner choose a trained agent without a single target reward. On new lags of Repeat Previous, choosing by freedom cut regret by 69% against choosing by regret on the trained lag. On new tasks of the delayed cue, it cut regret by 58%.

I have claimed elsewhere that an embodied learner should choose the correct policy with the most completions in its embodied language (Bennett, 2025). In every experiment here, the better agent was the one with the most freedom.

Acknowledgments and Disclosure of Funding

No external funding supported this study, and I declare no competing interests. The repository at `https://github.com/ViscousLemming/Technical-Appendices/tree/main/Papers/Regret_Is_Lost_Freedom` holds the code for all four environments, the plans fixed in advance, every selection record, the per-candidate tables of every replication, and the scripts for regenerating every number and figure in this paper.

Appendix A. Further results and proofs

A.1 Proofs from the main text

Proof of Lemma 3.1 If $y_i = 1$, then $V_i^\star - V_i = p_i - q_C p_i - (1 - q_C)(1 - p_i) = (1 - q_C)(2p_i - 1) = (1 - q_C)\, 2m_i$, which is the success case. Dividing by $V_i^\star = \frac{1}{2} + m_i$ gives $\rho_i(1 - q_C)$, which is the normalised-regret case. Scaling the success case by s_i gives the return case. The case $y_i = 0$ is symmetric, with q_C in place of $1 - q_C$. ■

Proof of Theorem 4.2 A mixture of decision rules has loss equal to the same mixture of their losses, so no mixture beats the best single rule. The first equality then follows from Lemma 3.1. The second equality holds because each $g \circ M$ is a table π_g. By the proof of Corollary 3.6, the freedom lost by π_g equals the loss of $g \circ M$.

Every $g \in G$ gives one prediction for each group. On group C, a prediction of 0 loses A_C, while a prediction of 1 loses B_C. Either loss is at least $\min\{A_C, B_C\}$. Summing over groups gives $D_G(M) \geq K_\ell(M)$. Equality holds if and only if some $g \in G$ attains $\min\{A_C, B_C\}$ for every group at once, which is the stated condition. The set G_{tab} contains a table that gives every group the prediction with the greater weight, so with G_{tab} the freedom lost is $K_\ell(M)$. ■

Proof of Proposition 4.4 Part 1 follows by applying Theorem 4.2 to each task T and averaging the least losses under Q. For part 2, I use a statement that contains the predictions of one rule g_T for each task T. Each demand concerns the groups of one task, and I can choose each task's rule separately. By Theorem 4.2, the least total freedom lost is therefore the sum of the least losses for individual tasks, weighted by $Q(T)$. This sum is $D_Q(M)$. The survival probability follows from Proposition 2.6 with $w_{i,T} = Q(T)\mu_i\ell_{i,T}$.

For part 3, all loss weights are one and four hidden classes are equally likely. Each class is determined by the history. The three tasks T_0, T_1 and T_2 split the classes as $\{1,2\}|\{3,4\}$, $\{1,3\}|\{2,4\}$ and $\{1,4\}|\{2,3\}$, respectively. Memory A holds only the information needed for T_0, so for each task the groups are $\{1,2\}$ and $\{3,4\}$. Under memory B, classes 1 and 4 share a state and the rest are separate. Then $D_{T_0}(A) = 0 < \frac{1}{4} = D_{T_0}(B)$. Under Q uniform on the three tasks, however, $D_Q(A) = \frac{1}{3} > \frac{1}{6} = D_Q(B)$. ■

Proof of Theorem 4.5 I treat each history as a state of a Markov decision process. The equality follows from the performance difference lemma of Kakade and Langford (2002), applied to π and the best policy that depends on the whole history. At step t, the agent uses one decision rule from G_t, or a mixture of such rules. Under d_t^π, every such rule has expected shortfall at least $D_t(\pi)$. A mixture averages these shortfalls, so it satisfies the same bound. Summing over steps gives the inequality. With two actions, the shortfall of an action is $|Q_t^\star(h,1) - Q_t^\star(h,0)|$ when the action differs from the best action, and zero otherwise. This is the loss of Definition 4.1 with that weight. ■

Proof of Proposition 4.6 Given h, a deterministic prediction a is wrong with probability p if $a = 0$, and with probability $1 - p$ if $a = 1$. In both cases, this probability is $\min\{p, 1-p\} + |2p-1|\mathbf{1}\{a \neq y\}$. After multiplying by s and taking expectations, I add and subtract $D_G^s(M)$ to obtain the three terms. The first two terms are nonnegative. The term labelled left unused is nonnegative because a comes from a decision rule in G, and $D_G^s(M)$ is the minimum over those rules. ■

Proof of Proposition 4.7 For each decision rule, summing the error bounds for individual points with weights μ_i shows that the estimated and true losses differ by at most ε. Their minima over G therefore also differ by at most ε. A memory $M^\star$ minimises D_G over $\mathcal{M}$. Then $D_G(\hat{M}) \leq \hat{D}_G(\hat{M}) + \varepsilon \leq \hat{D}_G(M^\star) + \varepsilon \leq D_G(M^\star) + 2\varepsilon$. The first and last inequalities use the error bound. The middle inequality uses the choice of $\hat{M}$ as a minimiser of $\hat{D}_G$.

For the bounds on ε_i, suppose first that p_i and $\hat{p}_i$ lie on the same side of $\frac{1}{2}$. The best predictions then agree, and ε_i is the change in the weight. That change is at most $2|\hat{p}_i - p_i|$ for $|2p-1|$, and at most $4|\hat{p}_i - p_i|$ for ρ, whose slope has magnitude at most 4. If p_i and $\hat{p}_i$ lie on opposite sides of $\frac{1}{2}$, ε_i is at most the larger of the two weights. Each weight is zero at $\frac{1}{2}$, so its slope bound limits it to that bound times the distance from its probability to $\frac{1}{2}$. That distance is at most $|\hat{p}_i - p_i|$. Multiplying the success bound by s_i gives the return bound. ■

A.2 Unweighted forgetting on the informative region

Corollary A.1 (Informative region) *For a fixed $0 < \gamma \leq \frac{1}{2}$, the informative region is $\mathcal{X}_\gamma := \{i : m_i \geq \gamma\}$. Within this region, $K_\gamma(M)$ is the least total probability $\sum_{i \in B \cap \mathcal{X}_\gamma} \mu_i$ that must be deleted to leave y constant on $(C \cap \mathcal{X}_\gamma) \setminus B$ for every group. With $c(\gamma) = 4\gamma/(1+2\gamma)$, the least normalised regret on this region satisfies*

$$c(\gamma) K_\gamma(M) \;\leq\; \inf_{\pi \text{ uses only } M} \sum_{i \in \mathcal{X}_\gamma} \mu_i \delta_i(\pi) \;\leq\; K_\gamma(M).$$

Proof On $\mathcal{X}_\gamma$, every weight satisfies $\rho_i \in [c(\gamma), 1]$. For each allowed deletion, its weighted cost lies between these same multiples of its total probability. By Theorem 3.3 on the restricted support, the least normalised regret equals the weighted forgetting cost. After minimisation over the same allowed deletions, this cost therefore lies between $c(\gamma)K_\gamma(M)$ and $K_\gamma(M)$. ■

Interpretation. On points whose margin is at least γ, every normalised-regret weight lies between $c(\gamma)$ and one. So on those points, the least normalised regret is within a factor $c(\gamma)$ of the least probability that must be deleted. Unweighted forgetting therefore approximates regret wherever the outcome is far from a coin flip.

A.3 More than two possible values

Theorem A.2 (More than two possible values) *Each point has a best prediction $y_i \in \{1, \ldots, k\}$. A policy that uses only M assigns a distribution q_C over prediction values to each group. At point $i \in C$, its loss is $\ell_i(1 - q_C^{(y_i)})$. For each group and prediction value, $A_C^{(j)} := \sum_{i \in C,\, y_i = j} \mu_i \ell_i$ is the total weight of points with that best prediction. The least average loss among policies that use only this memory is*

$$\inf_{\pi \text{ uses only } M} \sum_i \mu_i \operatorname{loss}_i(\pi) = \sum_{C \in \mathcal{C}_M} \Big(\sum_j A_C^{(j)} - \max_j A_C^{(j)} \Big).$$

This value is the least weight that must be deleted to leave one best prediction in each group. It is also the weighted Bayes risk of the grouping.

Definition 4.1 and Theorem 4.2 extend to every finite set of prediction values by the same proofs. Each demand then omits every program that predicts wrongly for its group. This Bayes risk replaces K_ℓ, and equality holds when some decision rule gives every group a prediction of greatest weight.

Proof The loss on group C is $\sum_j A_C^{(j)} - \sum_j A_C^{(j)} q_C^{(j)}$. This loss is least when q_C puts all probability on a prediction with greatest weight. A policy can choose a minimising prediction separately for each group, so these minima add to give its least total loss. I leave one best prediction in a group by deleting every point whose best prediction differs from the one left. The deleted weight is least when the prediction left has the greatest total weight. ■

A.4 A regret order on memories

Proposition A.3 (Ordering memories by regret) *I write $M_1 \preceq M_2$ when $K_\ell(M_1) \geq K_\ell(M_2)$. This is a preorder. Each memory is related to itself. If one memory is related to a second, and that second is related to a third, then the first is related to the third. If every group under M_2 is contained in a group under M_1, then $K_\ell(M_2) \leq K_\ell(M_1)$. The inequality is strict when some group under M_1 splits under M_2 into smaller groups, and the prediction with the greater weight is 1 in one of them and 0 in another.*

Proof For nonnegative reals, $\min\{a+a', b+b'\} \geq \min\{a,b\} + \min\{a',b'\}$, so splitting a group never raises its contribution to the sum. If $a > b$ and $a' < b'$, the inequality is strict, because $a+a'$ and $b+b'$ both exceed $b+a'$. Take $a = A_{C'}$, $b = B_{C'}$, $a' = A_{C''}$ and $b' = B_{C''}$ for two parts C' and C'' of the split group, with the prediction 1 heavier in C' and the prediction 0 heavier in C''. ■

Interpretation. Splitting groups never adds irreducible regret, and splitting a group into parts with different heavier predictions removes some. So a memory that distinguishes more histories never has a larger K_ℓ.

Theorem A.4 (When no freedom is lost) *The set $I_+ := \{i : \mu_i \ell_i > 0\}$ contains the indices of points with positive weight. The partition $\mathcal{P}_+^\star$ divides I_+ into the nonempty sets $\{i \in I_+ : T_i = t,\ y_i = b\}$ over tasks t and predictions b. Within each part, all points have the same task and the same best prediction. Then $K_\ell(M) = 0$ if and only if the groups defined by M, restricted to I_+, refine $\mathcal{P}_+^\star$. Here, refinement means that each nonempty restricted group is contained in one part of this partition. The partition $\mathcal{P}_+^\star$ is the unique coarsest partition of I_+ with zero cost among those that separate points with different tasks.*

Proof A group has zero cost if and only if all its positive-weight points share a best prediction. No group contains points of two tasks. So a memory has zero cost if and only if its groups, restricted to I_+, refine $\mathcal{P}_+^\star$. All points of each block of $\mathcal{P}_+^\star$ share a task and a best prediction, so each block has zero cost. Every zero-cost partition of these positive-weight points that separates tasks must keep points with different tasks or best predictions in separate parts. Such a partition therefore refines $\mathcal{P}_+^\star$, so $\mathcal{P}_+^\star$ is the unique coarsest such partition. ■

Interpretation. A memory loses no freedom if and only if no group contains two positive-weight points with different best predictions. A memory assigns one state to a history for every task at once. So when one history has different best predictions for different tasks, the coarsest memory with zero cost may have to be strictly finer than $\mathcal{P}_+^\star$.

A.5 Stability under estimation error

Proposition A.5 (Effect of estimation error) *Suppose ρ_i is replaced by $\hat{\rho}_i$, and the best predictions agree. Then*

$$|K_{\hat{\rho}}(M) - K_\rho(M)| \leq \sum_i \mu_i |\hat{\rho}_i - \rho_i| \leq 4 \sum_i \mu_i |\hat{p}_i - p_i|.$$

Proof For each group, $|\min\{a', b'\} - \min\{a, b\}| \le |a' - a| + |b' - b|$. The best predictions agree, so each point's weight is included in the same prediction's total before and after estimation. These bounds give the first inequality in the proposition when summed over groups. For $p > \frac{1}{2}$, the weight $\rho = (2p-1)/p$ has slope $1/p^2 \le 4$, and for $p < \frac{1}{2}$, the weight $\rho = (1-2p)/(1-p)$ has slope of size $1/(1-p)^2 \le 4$. Thus each point's weight changes by at most four times the error in its estimated probability. The average over points gives the second inequality. The case where the best predictions differ is part of Proposition 4.7. ■

Interpretation. Errors in the estimated probabilities move K_ρ by at most four times their average size. So a teacher whose probabilities are close to the truth gives a close estimate of K_ρ.

A.6 The measurable version

Proposition A.6 (Measurable reduction) *The history-task points form a probability space $(\mathcal{X}, \Sigma, \mu)$. The function $\zeta := (M(h), T)$ is measurable and takes values in a standard Borel space. The functions y and ℓ are measurable and integrable, with $\ell \ge 0$. Conditional on memory and task, always predicting zero or one has expected loss $a(z) := \mathbb{E}[\ell \mathbf{1}\{y = 1\} \mid \zeta = z]$ or $b(z) := \mathbb{E}[\ell \mathbf{1}\{y = 0\} \mid \zeta = z]$, respectively. The least expected loss over measurable prediction probabilities is*

$$\inf_{q \text{ measurable}} \mathbb{E}\big[\ell\big((1 - q(\zeta))\mathbf{1}\{y = 1\} + q(\zeta)\mathbf{1}\{y = 0\}\big)\big] = \mathbb{E}\big[\min\{a(\zeta), b(\zeta)\}\big],$$

attained by $q^\star(z) = \mathbf{1}\{a(z) > b(z)\}$.

Proof I condition on ζ and then average to obtain $\mathbb{E}[a(\zeta)(1 - q(\zeta)) + b(\zeta)q(\zeta)]$. At each z, this expression has constant slope in $q(z)$, so its minimum at z is $\min\{a(z), b(z)\}$, attained by $q^\star$. This choice puts all probability on a prediction of least loss. Under the standard Borel assumption, a and b have measurable versions, so this pointwise choice is measurable. ■

Interpretation. The reduction holds for memories with continuous states, such as the hidden state of a recurrent network. On a finite support, ζ takes one value per group and the right side is $\sum_C \min\{A_C, B_C\}$.

Appendix B. The bridge in Stack-Theoretic terms

B.1 Generalisation probability from regret

Corollary B.1 (Generalisation probability) *Take the demands u_i of Corollary 3.6 with $w_i = \mu_i \rho_i$. Suppose the world imposes the demand of each point x_i independently with probability $r_i = 1 - e^{-\mu_i \rho_i}$. Under this prior, the best policy that uses only M survives with probability $e^{-K_\rho(M)}$.*

Proof A statement ϑ survives when every demand that the world imposes lies in $\mathrm{Ext}(\vartheta)$. Survival therefore requires that none of the demands ruled out by this statement is imposed. By independence, as in Proposition 2.6, that event has probability

$\prod_{i:\, u_i \notin \mathrm{Ext}(\vartheta)} (1 - r_i) = \exp(-\sum_{i:\, u_i \notin \mathrm{Ext}(\vartheta)} w_i)$. By Corollary 3.6, the demands that the best table rules out have total weight $K_\rho(M)$. Substitution of this total weight into the probability expression gives the claimed survival probability. ■

The prior is a premise of the corollary. By adopting it, I commit to a view of which unseen demands the world is likely to impose. Under another prior, the best policy for the same memory survives with another probability. With $K_\rho = 0.02$, the best policy that uses only M survives with probability $e^{-0.02} \approx 0.98$. With $K_\rho = 0.20$, the probability falls to $e^{-0.20} \approx 0.82$.

B.2 When predictions for different groups are linked

Example B.2 (Linked predictions for two groups) *Take four points under one task with probabilities* $\mu = (0.3, 0.2, 0.1, 0.4)$*, best predictions* $y = (1, 0, 1, 0)$ *and* $p_i \in \{0, 1\}$*, so* $\rho_i = 1$ *and* $w_i = \mu_i$ *for every point. Take a memory with groups* $C_1 = \{x_1, x_2\}$ *and* $C_2 = \{x_3, x_4\}$*. Then* $A_{C_1} = 0.3$*,* $B_{C_1} = 0.2$*,* $A_{C_2} = 0.1$ *and* $B_{C_2} = 0.4$*, so* $K_\rho(M) = 0.3$*. This is the freedom lost by the table* $\{a_{C_1,1}, a_{C_2,0}\}$*. Now allow only the two constant decision rules, whose tables are* $\{a_{C_1,1}, a_{C_2,1}\}$ *and* $\{a_{C_1,0}, a_{C_2,0}\}$*. The table that predicts* 1 *for both groups rules out the demands of* x_2 *and* x_4*, of weight* 0.6*. The table that predicts* 0 *for both groups rules out the demands of* x_1 *and* x_3*, of weight* 0.4*. So for the same memory, the freedom lost with this set of rules is* 0.4*, above* $K_\rho(M) = 0.3$*.*

Interpretation. A constant decision rule gives the same prediction for every group, so the prediction for C_1 determines the prediction for C_2. Predictions for different groups are linked in the same way under a linear decision rule for a continuous memory. So restricting the decision rules raised the least regret from 0.3 to 0.4 with no change in memory.

B.3 What weighted freedom does not imply

By the Sufficiency and Necessity theorems, unweighted freedom is optimal under maximal ignorance in every embodied language (Bennett, 2025). Weighted freedom is optimal only under the prior that defines its weights. Adopting a prior commits the agent to a view of which unseen demands the world is likely to impose. Re-encoding the agent changes which unseen outputs bear the weights. I use weights because regret is defined with them, and the identity of Corollary 3.6 must match regret. The unknown-task result of Proposition 4.4 is unweighted when the support probabilities are equal, every loss weight is one and the prior over tasks is uniform.

B.4 Enactive general reinforcement learning

Stack Theory was first applied to reinforcement learning in Enactive General Reinforcement Learning (EGRL) (Bennett, 2025). EGRL divides interaction histories into parts that pass or fail a yes-or-no reward rule. It ranks allowed pairs of policies by a score based on freedom, and it may delete outputs that contradict otherwise good policies. The deletion in Definition 3.2 is that selective deletion, applied to a support. Nothing in the main text assumes EGRL beyond the definitions of Section 2.

Appendix C. Environments, plans and procedures

C.1 Plans fixed in advance

Pilots. No random seed used in a pilot was used in the confirmatory replications. The first E1 pilot had target delays of 12 to 28 steps, and its rare-regime share rose from 0.10 at source to 0.25 at target. In that pilot, source validation matched the best agent in hindsight. In the plan fixed in advance, I therefore moved the target delays to 40, 60 and 80 steps, and the rare-regime shares to 0.05 at source and 0.30 at target. For each experiment I fixed conditions that its pilot had to meet. The E1 pilot had to show an average spread of at least 0.20 in target success after refitting, where the spread is the range across the candidates of one replication. In the second E1 pilot, the spread was 0.27 at common-regime reliability 0.95. At reliability 0.45 it was 0.13, so the two spreads averaged 0.20 and met the condition. The E2 pilot had to show that at least a quarter of candidates reached source success 0.9, and that target success at hallway length 40 had a range of at least 0.3 across candidates. Both conditions held. In the E2 pilot, I found that the gains of layer normalisation were zero at initialisation because of a bug, so the transformer candidates could not learn. I fixed it before the plan was finalised. For E3, I piloted RockSample(5,5) with the POBAX configuration, which has size 5, five rocks, half-efficiency distance 20, rewards ±10 and exit reward 10. Within a hundred updates, every agent learned to leave at once, without checking or sampling a single rock. The pilot of the fallback had two conditions. Across candidates, the number of cue samples in the risky regime had to differ by at least one, and the target return after refitting had to span at least one unit. In the pilot replication, the samples ranged from 0 to 11.5. The returns ranged from 1.05 to 5.21. In the E4 pilot I ran two replications on random seeds that were not used in E4 or E5. The first E4 pilot condition required that at least a quarter of candidates reach source return 0.9. In the pilot, the share that reached it was 0.51. The second condition required an average spread of at least 0.15 in success on the new lags after refitting, across the candidates of a replication. The spread in the pilot was 0.54. After the pilot, I held to 300 updates, because width-32 recurrent networks still improved after update 150. In the pilot, some candidates stayed below a source return of 0.95, so I held to the margin of 0.10 for removing candidates.

Amendments. I made two amendments, each because a machine failed and neither because of a target outcome. The E1 selection stages were split between a two-core cloud machine and a four-core virtual machine on my laptop. The virtual machine ran every E2 and E3 selection stage. The virtual machine disconnected twice and the cloud machine restarted once. By then, the E2 evaluation stage had been allowed to begin. Before any E1 or E3 evaluation stage ran, I recorded which replications each experiment would evaluate. In E1, evaluation would include every replication whose selection record had been saved by the time the remaining work finished. In E3, it would include all 24 replications. Inclusion depended only on the order in which replications finished. In the end, all 192 E1 selection records were saved. The cloud machine ran the replications with index 0 or 2 modulo 3 and the replications from 37 to 82 with index 1 modulo 3. The virtual machine ran the rest. Their selection records and trained agents were copied to the cloud machine before the E1 evaluation stage was allowed to begin. All 24 E3 replications and all 32 E2 replications ran on the virtual machine. A partial E3 run on the cloud machine was stopped and discarded

Table 2: Training and evaluation settings. Numbers of episodes are per replication unless marked per candidate.

Setting	E1	E2 and E3	E4 and E5
PPO updates	150	300	300
Episodes per update	192	64	64
Epochs	2	3	3
Minibatch size	64	32	32
Learning rate	2.5×10^{-3}	10^{-3}	10^{-3}
Entropy coefficient	0.01	0.01	0.01
Discount	0.99	0.99	0.99
Generalised advantage parameter	0.95	0.95	0.95
Clipping	0.2	0.2	0.2
Value coefficient	0.5	0.5	0.5
Gradient norm	1	0.5	0.5
Versions saved at updates	75 and 150	150 and 300	150 and 300
Source episodes	3000 for the rounding, 3000 for teachers and 2400 for source validation	256 per candidate	512
Target sample	1600 histories	384 episodes per candidate	512 episodes
Fresh target episodes	2400	600 per candidate	1000

before any of its selection records were saved. E4 and E5 ran as 16 processes directly on the laptop that hosted the virtual machine. A first launch of E5 inside the virtual machine stopped after six agents of one replication had trained, because the virtual machine ends background processes. Before any E5 selection record was saved, I recorded the failure, set the six agents aside unused and restarted E5 from the beginning on the laptop.

C.2 E1

Table 2 contains the training settings and the numbers of episodes in every experiment.

Successive prediction steps are three steps apart. In the part with longer delays, the common regime has one of four cue reliabilities, 0.95, 0.75, 0.60 or 0.45. Each reliability defines a set of 36 replications. Within a replication, the reliability varies randomly by up to 0.02. In the rare regime, cue reliability is drawn per replication from $[0.96, 0.995]$. The part with new tasks runs at common-regime reliability 0.60. Observations are 16-dimensional one-hot vectors with two regime flags, four cue symbols, five distractor symbols, three flags indicating the task, a start flag and a padding flag. Each of the three tasks divides the four classes two against two. The task order, and the choice of which pair receives the positive label, are drawn at random in each replication.

In the part with longer delays, the pool includes every combination of the following settings. The layer is GRU or LSTM, and the width is 16 or 64. The training delays are 3

to 7, 3 to 14 or 3 to 28 steps, and the rare-regime share is its natural 5% or raised to 50%. With versions saved at updates 75 and 150, each replication has 48 agents. The memory is the hidden state one step before the first prediction step. Each agent's memory is rounded to six states by k-means on standardised source memories. In the five roundings, the six states are divided between the regimes as $5+1$, $4+2$, $3+3$, $2+4$ and $1+5$, so each replication has 240 six-state memories. In the part with new tasks, agents differ in whether they have an auxiliary head that predicts the most recent cue (Jaderberg et al., 2017). This setting replaces the range of training delays. Each replication there has 32 agents. Rounding them in five ways gives 160 six-state memories. A candidate stays in the pool when the source success of its own policy is within 0.10 of the best in its replication.

Agents use a GRU or LSTM layer followed by actor and critic heads with one hidden layer of the same width and a hyperbolic tangent. I train them by PPO on predictions at the prediction steps. The auxiliary head in the part with new tasks predicts the most recent cue symbol, and its loss has weight 0.5. The primary teacher embeds each step with two layers of 64 rectified units, sums the embeddings, appends the regime flag and outputs four class logits through two layers. The GRU teacher has width 64. Both train for 40 epochs with Adam at learning rate 3×10^{-3}. The analytic posterior assumes that a cue shows the class with probability equal to the regime's reliability, and otherwise shows one of the other three classes at random. Each regime's reliability is estimated as the share of source cues equal to the class. For the test of Proposition 4.7, noise of standard deviation 0.5, 1 and 2 is added to the analytic log-posterior.

Linear decision rules are fitted by weighted logistic regression, using Newton steps with an ℓ_2 penalty of 10^{-3} on standardised memories. I measure the freedom lost by five-fold cross-validation on the target sample. I fit the rule on four folds, score its errors on the fifth, and repeat this for every fold. The nearest-neighbour rule is a vote over 15 neighbours, weighted by the teacher's weights. Teacher evaluation is the average success lost by the agent's own prediction probabilities against the teacher's best prediction. The average runs over the prediction steps of every history in the target sample. Deep embedded validation fits a logistic classifier on standardised memories to distinguish source from target, caps density ratios at 50 and uses the correction of You et al. (2019) described in Section 6. For thresholded confidence, the threshold is set so that the share of source predictions above it equals the source accuracy. Disagreement is the share of predictions on the target sample that differ from the majority of the pool.

C.3 E2 and E3

E2 episodes end after four times the length of the shortest path. E3 episodes end after four times that length plus 16 steps. In E3's ordinary regime, a sample of the cue is correct with probability 0.95 and a turn earns $+4$ or -0.1 as in POBAX. In the risky regime, a sample is correct with probability 0.70 and a turn earns $+8$ or -4. In E2, I train two seeds of each of the eight architectures. In E3, I train each architecture at the natural and at a raised share of the risky regime. With versions saved at updates 150 and 300, each replication has 32 candidates. A candidate stays in the pool when the source performance of its own policy is within 0.10 success of the best in its replication in E2 and within 1.0 return in E3. Recurrent candidates feed the observation to a GRU or LSTM layer whose output feeds

actor and critic heads with one hidden layer of 64 hyperbolic-tangent units. Transformer candidates embed the observation into 32 dimensions and apply two blocks and a final layer normalisation (Parisotto et al., 2020). Each block has layer normalisation, two-head causal attention over the last 8 or 32 steps with a learned bias per relative offset, and gated residual connections with gate bias 2. The memory is the policy head's input at the first step at the junction. At the first step at the junction, the turn comes from the refitted decision rule. An episode in which the agent never reaches the junction has a score of zero. I run each candidate in its own labelled source episodes, and its target sample consists of its own unlabelled target episodes. The source performance of each candidate's policy is measured on its source episodes. The E2 teacher embeds each step with two layers of 32 rectified units, max-pools the embeddings and outputs the goal through two layers. The E3 teacher sums the embeddings instead and appends the regime flag. Both train on the source histories of all candidates up to the first step at the junction. Teacher evaluation scores each episode that ends at the junction. The score uses the agent's own probabilities of turning north and south, renormalised over the two turns and weighted by the teacher's posterior. The score is in success for E2 and in return for E3. Disagreement compares each candidate's turn on 240 junction histories drawn from every candidate in the pool.

C.4 E4 and E5

Observations are one-hot suits, with no step counter. The agents are those of E2 and E3 with four actions. The four-card transformer has one block and an attention window of 4 steps. The memory is the policy head's input at every step. The 36 candidates per replication are two seeds of each of the nine architectures, with versions saved at updates 150 and 300. A candidate stays in the pool when its source return is within 0.10 of the best in its replication. A transformer with one block of window w attends to the w most recent steps. A transformer with two blocks of window w attends to the $2w - 1$ most recent steps. The two-block transformers of windows 8 and 32 therefore reach back 15 and 63 cards. Training batches come from a NumPy implementation of Repeat Previous. In every replication, I check the implementation against `popgym` 1.0.7 on 200 seeded episodes of `RepeatPrevious(1,4)`. The check compares observations and rewards under a fixed random action sequence. On lags 1 to 16, the suit computed by the implementation must earn the package's reward at every step at which that lag can be evaluated. The package deals the source episodes excluded from training and the target sample. It also deals and steps the fresh target episodes. The NumPy implementation computes only the correct suits. The agent's predictions never change observations. So one pass over an episode gives the memory, action probabilities and value at every step, using only current and earlier observations at each step. I checked this shortcut against stepping the agent. For lag k, a multinomial logistic regression predicts the suit of the kth most recent card from the memory at step t. The freedom lost d_k is its error on data excluded from fitting, scored over steps $t \geq k$ of the target sample. The regression gives each example unit weight and uses an ℓ_2 penalty of 10^{-3} on standardised memories. I use five-fold cross-validation over episodes. I fit on four folds, score errors on the fifth, and repeat this for every fold. All steps from one episode lie in one fold. Freedom over all tasks is the average of d_k over $k = 1, \ldots, 51$ with weight $52 - k$, the number of steps at which lag k can be evaluated. Regret on lag 4 is d_4. The confidence

scores are computed from the action probabilities at the steps of the target sample at which lag 4 can be evaluated. The threshold of thresholded confidence is fitted on source steps excluded from agent training. Network size is the number of weights in the whole network. The decision rule for each lag from 1 to 16 is refitted on the full target sample and stored in the selection record. For Kendall's τ_b, scores within 10^{-9} and outcomes within 10^{-12} are taken as tied. A score that is constant over the pool has $\tau = 0$. The selection stage of a replication took about two hours on one core, most of it fitting 51 decision rules for each of 36 candidates.

C.5 Selectors and deployments

Table 3 lists the selectors, and their variants are named in the tables of Section C.6. In E1, the deployed agent predicts from a table with one entry per memory state or from a linear layer. Either is refitted on the target sample with teacher labels. In a third deployment, the agent's own policy is scored unchanged. In E2 and E3, the deployed agent moves with its own policy. At the junction, the turn comes from its own policy or from a linear layer refitted on its target sample. In E4 and E5, the predictions at each lag come from a multinomial logistic regression refitted on the target sample. The outcome is the success of those predictions over the six new lags. In E1, the primary teacher is a set network with sum pooling over the episode's observations. The other two teachers are a GRU over the episode and the analytic posterior, whose cue reliabilities are estimated on source episodes. In E2, the teacher is a set network with max pooling, since the goal is known from one observation at the start. In E3, it is a set network with sum pooling, since the posterior depends on how many samples of each kind the agent took.

C.6 Full results

Tables 4 to 7 contain every selector's outcome and share of the gap closed in every experiment. Figure 5 shows the shares of the gap closed in E1 to E3 for both ways of deploying. Figure 6 shows how well each selector ranked the pools of E4 and E5. Figure 7 shows the samples of the cue taken by the agents that every E3 selector deployed.

C.7 Procedures and statistics

Every selection record contains the seed of its fresh episodes and a field recording that no target label or reward was used. E4 and E5 selection records contain the package version, the result of checking the NumPy implementation and a field recording that no selector used the list of deployed lags. Fresh target episodes are generated only inside the evaluation stage. I compute teacher error only after the evaluation stage is allowed to begin. It is the error on the target sample against the exact posterior of the generating process. In the analysis I recompute every selection from the saved scores. Online validation draws its n rewarded episodes per candidate from the fresh target episodes, so it has a slight advantage over selectors that use no target rewards.

Statistics. Intervals are percentile bootstrap intervals over replications with 50,000 resamples. Sign-flip tests are exact up to 24 replications. Above that, they are approximated with one million random flips, so the smallest attainable p is about 10^{-6}. The *bottom-quarter*

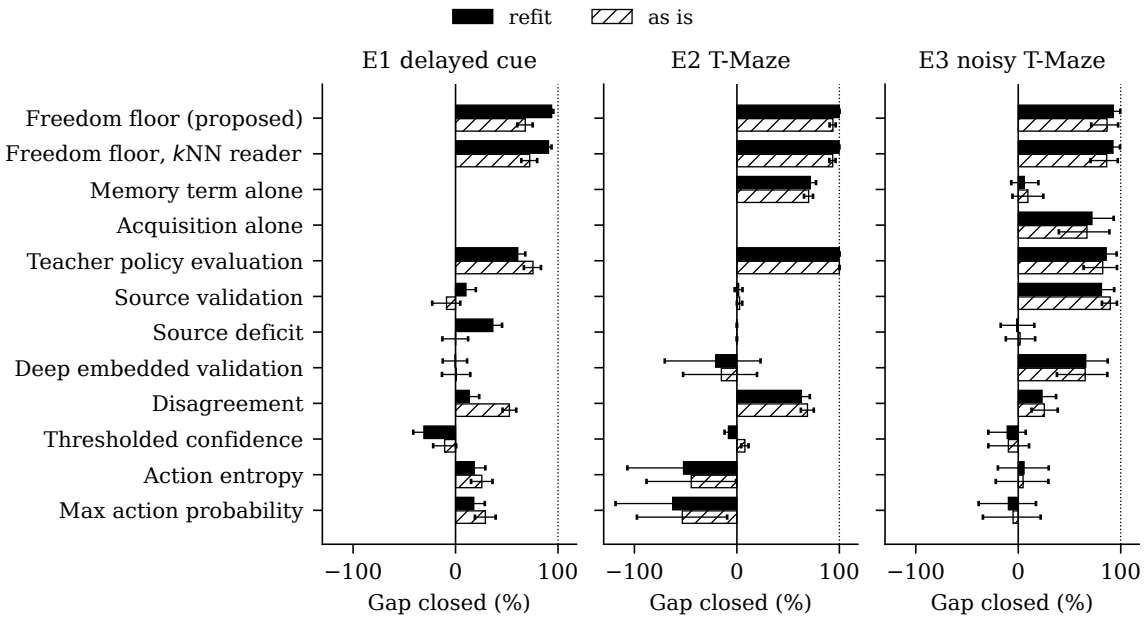


Figure 5: Share of the gap closed by each selector, pooled over replications. The gap runs from random choice (0%) to the best agent in hindsight (100%). Solid bars are after refitting the decision rule, and hatched bars are with the agent unchanged. Freedom means the freedom lost with a linear decision rule in E1, and the freedom selector of Section 5 in E2 and E3. In E2, the memory term only is the freedom lost with a linear decision rule on the episodes in which the agent reaches the junction. In E3, it is the freedom lost in memory. Bars below zero mark selectors that chose worse than random. Error bars are 95% bootstrap intervals over replications.

rate of a selector is the expected share of replications in which it deploys a candidate from the bottom quarter of the pool.

Ties. A selector's best score may be shared by several candidates, to within 10^{-12} in E1 to E3 and 10^{-9} in E4 and E5. The selector then deploys each of them with equal probability, and every outcome, bottom-quarter rate and paired difference is the expectation under that choice. The plans fixed in advance for E1 to E3 contain no rule for ties, and those for E4 and E5 contain the equal-chance rule. In the analysis of E1 to E3, I first broke ties by the lowest candidate index, which follows architecture and size. Under that rule, the smallest memories are deployed wherever many candidates share a score, as source validation's candidates do in E2. In the text and tables, tied candidates are deployed with equal chance. Table 8 lists every share of the gap closed that moves by two points or more between the two rules. The results of the proposed selectors after refitting do not move.

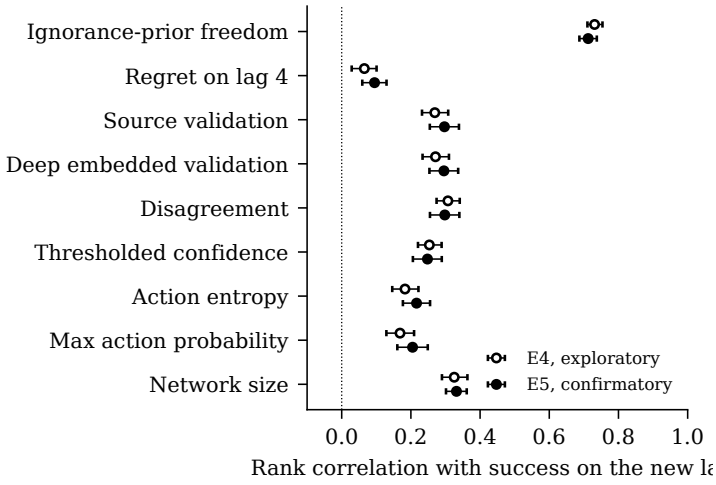


Figure 6: Repeat Previous. Mean rank correlation τ between each selector's score and the candidates' success on the new lags, with 95% bootstrap intervals over replications. Open markers show E4, and filled markers show E5. The ranking is exploratory in E4 and confirmatory in E5.

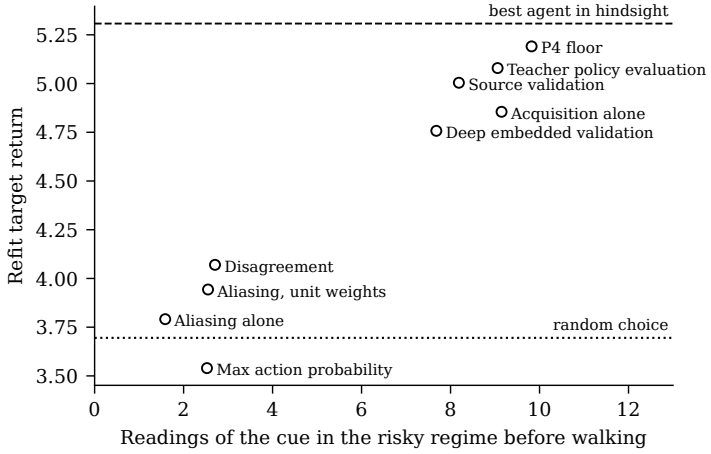


Figure 7: E3. Target return after refitting against the number of cue samples taken in the risky regime before walking, for the agents that each selector deployed, averaged over the 24 replications.

C.8 Further results

The auxiliary head. In the part of E1 with new tasks, some candidates trained an auxiliary head to predict the most recent cue. Their memories therefore hold the cue itself, at a small loss of success on the known task. Freedom deployed an agent with this head in 100% of replications. Regret on the known task did so in 16%. The head alone does not

explain freedom's lead on the new tasks, because random choice among the candidates with the head closed only 49% of the gap. Among the candidates with the head, freedom closed 91% of the gap. Regret on the known task closed 31%. Freedom beat it in all 48 replications.

E4's second hypothesis. E4's second hypothesis was fixed before E4 ran. For it, I compared the agent that freedom deployed with the agent that each of seven rivals deployed. Freedom beat thresholded confidence, disagreement and network size. It was indistinguishable from source validation, deep embedded validation, maximum action probability and action entropy (Table 10). The agents in that test were nearly identical. Source validation, maximum action probability and deep embedded validation deployed the same agent in all 32 E4 replications. That agent was a GRU or LSTM of width 128 in 31 of those replications. Freedom deployed a GRU or LSTM of width 128 in all 32. Apart from one replication, the success of the two agents on the new lags differed by at most 0.025. In that replication, source validation deployed a transformer that scored 0.422 on the new lags. Freedom's agent scored 0.897. The best agent in hindsight averaged only 0.008 above freedom's agent, so even a perfect selector had little room for improvement. I regard E4's second hypothesis as a poor test and replaced it with E5.

E5's secondary statistics. All 16 confirmatory tests of E5 held (Table 10). Without the four-card transformers, regret on lag 4 rose to $\tau = 0.44$. Freedom's ranking still had the higher τ in 30 of the 32 replications. The order was the same under three secondary statistics and under the same analysis applied to E4 after the fact (Tables 7 and 11). For one deployed agent per replication, freedom led source validation by only 0.005 in success (Table 11).

Knowing the tasks. No selector may use the deployed lags. In exploratory analyses on E5, a reference ranking by the freedom lost on those lags had $\tau = 0.98$. Freedom over lags 1 to 8 alone had $\tau = 0.98$. The prior of freedom over all tasks puts 71% of its weight on lags beyond 8. So memories that hold older cards score well under this prior, although no deployed lag concerns those cards. Freedom over all tasks is computed without the list of deployed lags, and it fell 0.27 short of that reference in τ. With the whole pool, its τ still exceeded every rival's in every replication.

Longer delays in E1. Source validation chose an agent trained on the shortest delays in 83% of replications. The best agent in hindsight was such an agent in only 23%. Freedom with success weights chose one in 26%. Freedom with success weights chose a memory from the bottom quarter of the pool in none of the 144 replications. Source validation did so in 22% of those replications. Teacher log loss closed 88% of the gap, and its difference from freedom with success weights was not significant (Table 9). For the agents' own memories with a refitted linear layer, every rival that needs no teacher closed at most 18% of the gap (Table 4). Freedom with a nearest-neighbour rule chose slightly worse agents than freedom with a linear decision rule, as predicted by Theorem 4.2 when the deployed decision rule is linear (Table 12). For agents shipped unchanged, the difference between teacher evaluation and freedom with a linear decision rule was not significant. After refitting the decision rule, freedom won in more replications than it lost to teacher evaluation (Tables 4 and 9).

The T-Maze. On average, 25.1 of the 32 candidates per replication stayed in the pool. Of those, 57% reached perfect target success once their decision rule for the junction was refitted.

So the outcome after refitting measures whether a selector avoids the other candidates. Source validation's top score was shared by 22.6 of the candidates on average, because they solved every source hallway. Maximum action probability closed −63% of the gap. Action entropy closed −52%. The freedom selector beat source validation and every rival that needs no teacher, and lost to none of them in any replication (Table 9). It beat the selector *memory term only*, winning in 29 of 32 replications without a loss. Within replications, it predicted each candidate's success after refitting with a median R^2 of 0.995 (Table 12). Teacher evaluation did better for agents shipped unchanged, as predicted in Section 5. For unchanged agents, it closed 100% of the gap. The freedom selector closed 94%. An agent shipped unchanged turns at the junction by its own policy, and that policy is outside the freedom selector's score. After refitting, the two tied in 31 of 32 replications.

The noisy T-Maze. The freedom selector beat the selector *memory term only* in 23 of 24 replications (Table 9). In the risky regime, the average agent in the pool sampled the cue 2.5 times. The selector *never-gained term only* cannot separate two agents that sample equally often. In the 6 replications where it and the freedom selector chose differently, the freedom selector chose the better agent in 5. Within replications, the freedom selector predicted each candidate's return after refitting with a median R^2 of 0.88. After the evaluation stage, I used the exact posterior to measure how far the refitted agents fell short of the return of an agent that observes the hidden variable. Most of that shortfall was freedom never gained (Table 12). Source validation closed 81% of the gap after refitting, because within replications the agents that sampled more tended to score more on source episodes. The rank correlation between samples and source return was positive in 17 of 24 replications. The freedom selector led source validation, but not significantly (Table 9). Deep embedded validation closed 66%. For agents shipped unchanged, teacher evaluation closed 82% of the gap. The freedom selector closed 87%. Contrary to my prediction, teacher evaluation lost on its own ground. Maximum action probability again chose worse than random. The selector *memory term only* did no better with return weights than with success weights or unit weights (Table 9).

Stakes and weights. Selection depended on the weights less than the theory allows. In E1 with new tasks, freedom over all tasks with unit weights did as well as with the weights (Table 5). In most E1 replications, freedom chose the same memories with success, normalised-regret and unit weights, and the shares of the gap closed were nearly equal (Table 4). The advantage of success weights over unit weights did not grow as the common regime became less reliable. Freedom with normalised-regret weights and freedom with success weights tied in most replications at every reliability level. I had predicted that normalised-regret weights would win on normalised regret and success weights on success, but no such crossover appeared (Table 12). How memory was shared between the regimes did depend on the weighting. On average, the memories that freedom with success weights chose had 2.92 of their six memory states in the rare regime. With unit weights the figure was 2.74. The best agents in hindsight had 2.78 in the rare regime. The memories chosen by source validation had only 1.99, the least memory for the rare regime. That regime is six times as frequent at target as at source. In E3, the selector *memory term only* chose the same agents with return and success weights in most replications, but with unit weights it chose other agents in most replications (Table 9). In these environments, the selectors

differed mainly in which histories they used and which decision rule they assumed. Once those two choices were fixed, the outcomes depended little on the weights.

Teacher error. By Proposition 4.7, the selection loss on the target sample is at most twice the teacher's error. In E1 the bound held in 100% of replications for every teacher. The primary teacher's error was 0.332. Its selection loss was only 0.0376. Adding noise to the analytic posterior raised its error to as much as 0.228. Its selection loss stayed at or below 0.0019 (Table 12). So the bound is loose in practice, because a teacher needs to be accurate only where memories differ. The least accurate teacher was the GRU teacher, which was trained on delays of at most 7 steps. It extrapolated poorly to a delay of 80 steps. With it, freedom with success weights closed 68% of the gap. The primary teacher discards the time of each step. With it, freedom with success weights closed 90%. In E2 the teacher's error was only 1.1×10^{-4}, because the goal is known from one observation at the start. In E3 the teacher's error was 1.98 return units. The freedom selector still closed 93% of the gap.

C.9 Paired tests and further statistics

Tables 9 and 10 contain the paired comparisons within replications. Tables 11 and 12 contain the intervals, fits and secondary comparisons summarised in Section 8.

Table 3: Selectors. In the column *target inputs*, yes means that the selector uses the unlabelled target sample, and *own episodes* means that it uses each candidate's own target episodes. In the column *teacher*, yes means that the selector uses teacher labels, and *posterior* means that every task is labelled from the teacher's probabilities for the hidden class. Only online validation uses target rewards. In the plans fixed in advance, the freedom selectors of E2 and E3 are called the P3 floor and the P4 floor.

Selector	Target inputs	Teacher	Role
Freedom with a table, $\hat{K}_\Delta$ and $\hat{K}_\rho$	yes	yes	Proposed, six-state memories (E1)
Freedom with a linear decision rule, $\hat{D}_{\text{lin}}$	yes	yes	Proposed, continuous memories (E1)
Freedom over all tasks, $\hat{D}_Q$	yes	posterior	Proposed, unknown tasks (E1)
Freedom over all tasks, every lag weighted by how often it can be evaluated	yes	no	Proposed, unknown tasks (E4, E5)
Freedom, $1 - \text{reach} + \text{reach} \cdot \hat{D}_{\text{lin}}$	own episodes	yes	Proposed, longer hallways (E2)
Freedom, never gained plus lost in memory plus lost on episodes in which the agent never reaches the junction	own episodes	yes	Proposed, more to sense (E3)
Freedom with a table, unit weights, $\hat{K}_1$	yes	yes	Without the weights
Freedom with a table, other teachers	yes	yes	GRU teacher, analytic posterior (E1)
Freedom over all tasks, unit weights	yes	posterior	Without the weights
Freedom over lags 1 to 8 or 1 to 16	yes	no	Narrower priors (E4, E5)
Memory term only, with each weight, and never-gained term only	own episodes	yes	Parts of the E2 and E3 selectors
Freedom with a nearest-neighbour rule	yes	yes	More flexible decision rules
Freedom on the deployed tasks	yes	posterior	Reference, computed with the deployed tasks (E1)
Freedom on the deployed lags	yes	no	Reference, computed with the deployed lags (E4, E5)
Regret on the known task, with each weight	yes	yes	Regret on the trained task (E1)
Regret on lag 4	yes	no	Regret on the trained task (E4, E5)
Teacher evaluation	yes	yes	Rival, return of the unchanged agent
Teacher log loss	yes	yes	Rival, fit of memory to teacher
Maximum action probability	yes	no	Hendrycks and Gimpel (2017)
Action entropy	yes	no	Morerio et al. (2018)
Thresholded confidence	yes	no	Garg et al. (2022)
Disagreement	yes	no	Jiang et al. (2022)
Deep embedded validation	yes	no	You et al. (2019)
Source validation	no	no	Standard practice
Freedom on source histories, and regret on the trained task on source histories	no	E1 to E3	Value of the target sample
Network size	no	no	Largest network (E4, E5)
Online validation	rewards	no	n rewarded episodes per candidate

Table 4: E1 with longer delays, 144 replications. Target success of the deployed agent, and the share of the gap closed by each selector. The gap runs from random choice to the best agent in hindsight. For memories rounded to six states, the predictions come from a table with one entry per state, refitted on the target sample. For the agents' own continuous memories, the predictions come from a refitted linear layer or from the agent's own policy.

Six-state memories	Refitted table	
Freedom, success weights	0.692	90
Freedom, normalised-regret weights	0.694	91
Freedom, unit weights	0.693	90
Freedom over all tasks	0.692	89
Freedom, GRU teacher	0.665	68
Freedom, analytic posterior	0.687	85
Teacher log loss	0.690	88
Freedom on source histories	0.603	17
Source validation	0.606	19
Random choice	0.582	0
Best agent in hindsight	0.705	100

Continuous memories	Refitted linear		Unchanged	
Freedom, linear decision rule	0.766	94	0.727	68
Freedom, nearest-neighbour rule	0.763	91	0.732	72
Teacher evaluation	0.733	61	0.736	76
Source validation	0.682	10	0.635	−9
Freedom on source histories	0.708	36	0.646	0
Deep embedded validation	0.671	−1	0.647	1
Disagreement	0.685	13	0.708	53
Thresholded confidence	0.640	−31	0.633	−11
Action entropy	0.690	18	0.676	26
Maximum action probability	0.689	18	0.680	29
Random choice	0.671	0	0.646	0
Best agent in hindsight	0.772	100	0.765	100

Table 5: E1 with new tasks, 48 replications. Memories are chosen with one task known. Deployment includes that task and two new tasks, and a table of predictions is refitted once all three are named. The reference row is a selector computed with the deployed tasks.

Selector	New tasks		Known task	
Freedom over all tasks	0.738	91	0.769	−7
Freedom over all tasks, unit weights	0.737	90	0.782	6
Regret on the known task, success weights	0.525	−36	0.877	100
Regret on the known task, normalised-regret weights	0.527	−35	0.877	100
Regret on the known task, unit weights	0.526	−35	0.877	100
Teacher log loss	0.647	37	0.876	99
Regret on the known task, source histories	0.521	−39	0.877	100
Source validation	0.539	−28	0.877	100
Freedom on the deployed tasks (reference)	0.742	93	0.777	2
Random choice	0.585	0	0.776	0
Best agent in hindsight	0.753	100	0.877	100

Table 6: E2 and E3, over 32 T-Maze replications and 24 noisy T-Maze replications. Mean target performance of the deployed agent, and the share of the gap closed by each selector. The gap runs from random choice to the best agent in hindsight. Performance is success in E2 and return in E3. Freedom is the freedom selector of each experiment (Section 5).

Selector	E2 T-Maze				E3 noisy T-Maze			
	Refitted		Unchanged		Refitted		Unchanged	
	success	%	success	%	return	%	return	%
Freedom	1.000	100	0.989	94	5.19	93	5.10	87
Freedom, nearest-neighbour rule	1.000	100	0.989	93	5.19	92	5.10	86
Memory term only	0.959	72	0.948	70	3.79	6	3.95	9
Never-gained term only	–	–	–	–	4.86	72	4.81	67
Memory term only, unit weights	–	–	–	–	3.94	15	4.08	18
Teacher evaluation	1.000	100	1.000	100	5.08	86	5.04	82
Source validation	0.859	2	0.831	3	5.00	81	5.15	90
Freedom on source histories	0.857	0	0.827	0	3.67	−2	3.84	2
Deep embedded validation	0.827	−21	0.801	−15	4.76	66	4.78	65
Disagreement	0.947	63	0.946	69	4.07	23	4.19	25
Thresholded confidence	0.845	−8	0.840	8	3.52	−11	3.67	−10
Action entropy	0.782	−52	0.750	−45	3.79	6	3.88	5
Maximum action probability	0.767	−63	0.735	−53	3.54	−10	3.74	−5
Random choice	0.857	0	0.827	0	3.70	0	3.81	0
Best agent in hindsight	1.000	100	1.000	100	5.31	100	5.30	100

Table 7: Repeat Previous, 32 replications in E4 and 32 in E5. For each selector, I report how often its chosen agent correctly predicts the suits of the second, third, fifth, sixth, seventh and eighth most recent cards. I refit a separate decision rule for each card position on the full target sample, using the agent's memory states as inputs. I measure success on fresh target episodes. The table also contains the share of the gap closed and the mean rank correlation τ between the selector's score and the candidates' success on the new lags. The gap runs from random choice to the best agent in hindsight. The ranking is exploratory in E4 and confirmatory in E5. Selectors below the first horizontal line are outside the groups of confirmatory tests. Freedom on the deployed lags is a reference that uses the deployed lags.

	E4			E5		
Selector	success	%	τ	success	%	τ
Freedom over all tasks	0.882	97	0.73	0.882	98	0.71
Regret on lag 4	0.619	13	0.07	0.614	13	0.09
Source validation	0.866	92	0.27	0.876	96	0.30
Deep embedded validation	0.866	92	0.27	0.876	96	0.30
Disagreement	0.776	63	0.31	0.729	49	0.30
Thresholded confidence	0.679	32	0.25	0.599	8	0.25
Action entropy	0.836	82	0.18	0.831	82	0.22
Maximum action probability	0.866	92	0.17	0.848	87	0.20
Network size	0.862	91	0.33	0.864	92	0.33
Regret on lag 4, source histories	0.622	14	0.07	0.614	13	0.10
Freedom, lags 1 to 16	0.883	98	0.85	0.883	98	0.82
Freedom, lags 1 to 8	0.890	100	0.98	0.889	100	0.98
Freedom on the deployed lags (reference)	0.890	100	0.98	0.889	100	0.98
Random choice	0.580	0		0.574	0	
Best agent in hindsight	0.890	100		0.889	100	

Table 8: Shares of the gap closed that move by at least two points between the two tie rules. Equal chance is the rule of the text and tables, and under it each tied candidate is deployed with equal probability. Under lowest index, the tied candidate listed first is deployed.

Env.	Deployment	Selector	Equal chance	Lowest index
E1	refitted linear	Thresholded confidence	−31%	−33%
E1	unchanged	Freedom on source histories	0%	−2%
E1 new	new tasks	Regret on the known task, success weights	−36%	−41%
E1 new	new tasks	Regret on the known task, normalised-regret weights	−35%	−40%
E1 new	new tasks	Regret on the known task, unit weights	−35%	−41%
E1 new	new tasks	Source validation	−28%	−32%
E1 new	new tasks	Regret on the known task, source histories	−39%	−46%
E2	refitted	Freedom, nearest-neighbour rule	100%	96%
E2	refitted	Memory term only	72%	36%
E2	refitted	Thresholded confidence	−8%	11%
E2	refitted	Disagreement	63%	14%
E2	refitted	Deep embedded validation	−21%	−10%
E2	refitted	Source validation	2%	36%
E2	refitted	Freedom on source histories	0%	10%
E2	unchanged	Freedom	94%	75%
E2	unchanged	Freedom, nearest-neighbour rule	93%	75%
E2	unchanged	Memory term only	70%	16%
E2	unchanged	Thresholded confidence	8%	12%
E2	unchanged	Disagreement	69%	15%
E2	unchanged	Deep embedded validation	−15%	−4%
E2	unchanged	Source validation	3%	26%
E2	unchanged	Freedom on source histories	0%	−7%
E3	refitted	Memory term only, unit weights	15%	17%
E3	refitted	Thresholded confidence	−11%	7%
E3	refitted	Disagreement	23%	26%
E3	refitted	Freedom on source histories	−2%	15%
E3	unchanged	Memory term only	9%	13%
E3	unchanged	Memory term only, unit weights	18%	21%
E3	unchanged	Thresholded confidence	−10%	14%
E3	unchanged	Freedom on source histories	2%	21%

Table 9: Paired comparisons within replications. Each row contains the mean difference within replications in the deployed agent's target outcome, its 95% bootstrap interval, the numbers of replications won, tied and lost, and the one-sided sign-flip p value. An asterisk marks a p value that is Holm-adjusted within the environment's group of confirmatory tests. E1 new is the part of E1 with new tasks, and freedom is the freedom selector of each experiment.

Env.	Comparison	Mean	95% interval	W/T/L	p
E1 new	Freedom − regret on the known task, new tasks*	0.212	[0.200, 0.223]	48/0/0	1.2×10^{-5}
E1 new	Regret on the known task − freedom, known task	0.108	[0.098, 0.118]	48/0/0	1.0×10^{-6}
E1	Freedom, table − source validation*	0.087	[0.076, 0.097]	127/7/10	1.2×10^{-5}
E1	Freedom, table − freedom on source histories*	0.089	[0.079, 0.100]	126/10/8	1.2×10^{-5}
E1	Freedom, table − teacher log loss*	0.002	[−0.002, 0.006]	44/76/24	0.325
E1	Freedom, table, success − unit weights*	−0.001	[−0.003, 0.002]	11/120/13	0.672
E1	Freedom, linear − source validation*	0.085	[0.073, 0.096]	123/13/8	1.2×10^{-5}
E1	Freedom, linear − maximum action probability*	0.077	[0.065, 0.089]	120/11/13	1.2×10^{-5}
E1	Freedom, linear − action entropy*	0.076	[0.064, 0.089]	118/14/12	1.2×10^{-5}
E1	Freedom, linear − thresholded confidence*	0.126	[0.113, 0.139]	135/5/4	1.2×10^{-5}
E1	Freedom, linear − disagreement*	0.081	[0.071, 0.092]	115/20/9	1.2×10^{-5}
E1	Freedom, linear − deep embedded validation*	0.095	[0.082, 0.109]	128/9/7	1.2×10^{-5}
E1	Freedom, linear − freedom on source histories*	0.058	[0.049, 0.068]	111/21/12	1.2×10^{-5}
E1	Teacher evaluation − freedom, linear, unchanged	0.009	[−0.003, 0.021]	60/47/37	0.080
E1	Freedom, linear − teacher evaluation, refitted	0.033	[0.026, 0.042]	80/47/17	1.0×10^{-6}
E2	Freedom − source validation*	0.141	[0.131, 0.152]	32/0/0	6.0×10^{-6}
E2	Freedom − maximum action probability*	0.233	[0.157, 0.311]	20/12/0	6.0×10^{-6}
E2	Freedom − action entropy*	0.218	[0.144, 0.296]	20/12/0	6.0×10^{-6}
E2	Freedom − thresholded confidence*	0.155	[0.140, 0.172]	32/0/0	6.0×10^{-6}
E2	Freedom − disagreement*	0.053	[0.038, 0.070]	29/3/0	6.0×10^{-6}
E2	Freedom − deep embedded validation*	0.173	[0.106, 0.252]	22/10/0	6.0×10^{-6}
E2	Teacher evaluation − freedom, unchanged	0.011	[0.006, 0.016]	23/9/0	1.0×10^{-6}
E2	Freedom − teacher evaluation, refitted	0.000	[0.000, 0.000]	0/31/1	1.000
E3	Freedom − memory term only	1.40	[1.07, 1.74]	23/0/1	1.2×10^{-7}
E3	Freedom − never-gained term only	0.34	[0.00, 0.75]	5/18/1	0.078
E3	Freedom − source validation*	0.19	[−0.10, 0.49]	8/12/4	0.123
E3	Freedom − maximum action probability*	1.65	[1.20, 2.08]	22/1/1	4.2×10^{-6}
E3	Freedom − action entropy*	1.40	[0.97, 1.84]	21/1/2	7.2×10^{-6}
E3	Freedom − thresholded confidence*	1.67	[1.35, 1.99]	23/1/0	7.2×10^{-7}
E3	Freedom − disagreement*	1.12	[0.82, 1.40]	22/1/1	4.8×10^{-6}
E3	Freedom − deep embedded validation*	0.43	[0.03, 0.87]	11/11/2	0.054
E3	Memory term only, return − success weights	−0.04	[−0.14, 0.04]	3/19/2	0.781
E3	Memory term only, return − unit weights	−0.15	[−0.48, 0.13]	5/9/10	0.813
E3	Teacher evaluation − freedom, unchanged	−0.06	[−0.16, 0.00]	0/20/4	1.000
E3	Freedom − teacher evaluation, refitted	0.11	[0.01, 0.24]	4/20/0	0.062

Table 10: Confirmatory tests on Repeat Previous, within replications. In E4, each test compares two agents' success on the new lags after refitting. One agent was deployed by freedom over all tasks and the other by the named selector. In E5, each test compares the rank correlations τ of the two selectors' scores with the candidates' success on the new lags. Each entry contains the mean difference within replications, the numbers of replications won, tied and lost, and the one-sided sign-flip p value. Each difference is freedom minus the named selector, and each p value is Holm-adjusted within the experiment's group of sixteen tests.

		E4, deployed agent			E5, ranking		
Pool	Freedom −	Mean	W/T/L	p	Mean	W/T/L	p
Whole pool	regret on lag 4	0.263	32/0/0	1.6×10^{-5}	0.62	32/0/0	1.6×10^{-5}
	source validation	0.017	16/5/11	0.610	0.42	32/0/0	1.6×10^{-5}
	deep embedded validation	0.017	16/5/11	0.610	0.42	32/0/0	1.6×10^{-5}
	disagreement	0.106	28/1/3	1.6×10^{-5}	0.41	32/0/0	1.6×10^{-5}
	thresholded confidence	0.203	29/0/3	1.6×10^{-5}	0.46	32/0/0	1.6×10^{-5}
	action entropy	0.047	15/6/11	0.330	0.50	32/0/0	1.6×10^{-5}
	maximum action probability	0.017	16/5/11	0.610	0.51	32/0/0	1.6×10^{-5}
	network size	0.021	29/0/3	1.6×10^{-5}	0.38	32/0/0	1.6×10^{-5}
Without four-card	regret on lag 4	0.016	28/0/4	5.5×10^{-5}	0.25	30/0/2	1.6×10^{-5}
	source validation	0.017	16/5/11	0.610	0.38	32/0/0	1.6×10^{-5}
	deep embedded validation	0.017	16/5/11	0.610	0.37	32/0/0	1.6×10^{-5}
	disagreement	0.017	23/2/7	0.001	0.31	32/0/0	1.6×10^{-5}
	thresholded confidence	0.018	18/5/9	0.004	0.32	31/0/1	1.6×10^{-5}
	action entropy	0.047	15/6/11	0.330	0.50	32/0/0	1.6×10^{-5}
	maximum action probability	0.017	16/5/11	0.610	0.49	32/0/0	1.6×10^{-5}
	network size	0.021	29/0/3	1.6×10^{-5}	0.48	32/0/0	1.6×10^{-5}

Table 11: Further statistics for Repeat Previous, E4 and E5, referred to in Section 8. Brackets contain 95% bootstrap intervals over replications. W/T/L stands for the replications won, tied and lost. The p values are from one-sided sign-flip tests, uncorrected unless marked Holm. Shares of the gap closed run from random choice (0%) to the best agent in hindsight (100%).

Env.	Quantity	Value
E4	Freedom minus regret on lag 4, success on the new lags	0.263 [0.257, 0.270]
E4	Regret on lag 4 minus freedom, success on lag 4	0.003 [0.002, 0.004], W/T/L 19/12/1
E4	Sum of freedom's advantage on the new lags and regret's advantage on lag 4	0.266 [0.260, 0.273]
E4	Without the four-card transformers, freedom minus regret on lag 4	0.016 [0.010, 0.022], Holm $p = 5.5 \times 10^{-5}$
E4	Mean τ computed after the fact, freedom and the seven rivals	0.73, 0.17 to 0.33
E4	Four-card transformer, success after refitting at lags 2, 3, 4 and 5	0.60, 0.57, 1.00, 0.26
E4	Share of replications in which regret on lag 4 deployed the four-card transformer	50%
E5	Mean τ of freedom	0.71 [0.69, 0.74]
E5	Spearman's rank correlation, freedom and the best rival	0.84, 0.52
E5	Rank correlation over lags 2, 3 and 5 to 16, freedom and the best rival	0.85, 0.45
E5	Shortfall of the best of the top three candidates from the best agent in hindsight, freedom and the others	0.0003, 0.006 to 0.055
E5	Largest uncorrected p in the three comparisons above	5.8×10^{-5}
E5	One deployed agent per replication, freedom minus regret on lag 4	0.267 [0.260, 0.275], W/T/L 32/0/0
E5	One deployed agent per replication, freedom minus source validation	0.005, W/T/L 21/3/8

Table 12: Further statistics for E1 to E3, with the conventions of Table 11.

Env.	Quantity	Value
E1 new	Sum of freedom's advantage in success on the new tasks and regret's advantage on the known task	0.321 [0.302, 0.338]
E1	Slope of realised regret after refitting on the estimated freedom lost with a linear decision rule, within replications	1.26 [1.21, 1.31]
E1	Median R^2 and mean absolute error of that fit	0.919, 0.100
E1	Linear decision rule minus nearest-neighbour rule, success of the deployed agent after refitting	0.003 [0.000, 0.007], W/T/L 24/96/24, $p = 0.035$
E1	Advantage of freedom with a linear decision rule over teacher evaluation, refitted deployment minus unchanged deployment	0.042 [0.030, 0.055]
E1	Trend of the advantage of success over unit weights with common-regime reliability	$r = 0.027$, $p = 0.372$
E1	Fewest replications, over the four reliability levels, in which normalised-regret and success weights tied	32 of 36
E1	Rare-regime memory states, success weights minus unit weights	0.18 [0.08, 0.29], $p = 2.2 \times 10^{-4}$
E1	Primary teacher, error and selection loss	0.332, 0.0376
E1	Analytic posterior, error	0.002
E1	Analytic posterior with noise 0.5, 1 and 2, error	0.063, 0.124, 0.228
E1	The same teachers, selection loss	0.0001, 0.0005, 0.0019
E1	Online validation at 32 and 64 episodes per candidate, share of the gap closed with a refitted table	86%, 92%
E1	Online validation at 8 episodes per candidate, share of the gap closed for unchanged agents	76%
E2	Freedom minus the memory term only, success after refitting	0.041 [0.030, 0.052], W/T/L 29/3/0, $p = 2.0 \times 10^{-6}$
E2	Bottom-quarter rate of maximum action probability and of action entropy	47%, 44%
E2	Slope of realised on predicted success after refitting, within replications	0.99 [0.96, 1.01]
E2	Median R^2 and mean absolute error of that fit	0.995, 0.012
E2	Replications in which freedom with the nearest-neighbour and linear decision rules chose alike	31 of 32
E2	Difference after refitting between freedom and teacher evaluation in the untied replication	0.0003
E2	Source validation, share of the gap closed at hallways 20, 40 and 80	−1%, 6%, 1%
E2	Freedom, share of the gap closed at hallway 80	100%
E2	Online validation at 16 and 32 episodes per candidate, share of the gap closed after refitting	98%, 100%
E3	Share of the gap closed by freedom	93% [82%, 100%]
E3	Samples of the cue in the risky regime before walking, best in hindsight, pool, freedom, memory term only and never-gained term only	10.1, 2.5, 9.8, 1.6, 9.1
E3	Share of the gap closed by the memory term only	6% [−7%, 20%]
E3	Slope of realised on predicted return after refitting, within replications	1.17 [1.05, 1.33]
E3	Median R^2 and mean absolute error of that fit, in return units	0.88, 0.65
E3	Shortfall of refitted agents from the return of an agent that observes the hidden variable, exact posterior	2.64
E3	The shortfall never gained, lost in memory and left unused	1.61, 0.80, 0.23
E3	The teacher's estimates of the never-gained and memory terms	1.39, 0.59
E3	Median rank correlation of risky-regime samples of the cue with source return	0.23, positive in 17 of 24 replications
E3	Online validation at 32 and 64 episodes per candidate, share of the gap closed after refitting	85%, 94%

References

Guillaume Alain and Yoshua Bengio. Understanding intermediate layers using linear classifier probes. In *International Conference on Learning Representations, Workshop Track*, 2017. URL https://mlanthology.org/iclr/2017/alain2017iclr-understanding/.

Ankesh Anand, Evan Racah, Sherjil Ozair, Yoshua Bengio, Marc-Alexandre Côté, and R. Devon Hjelm. Unsupervised state representation learning in Atari. In *Advances in Neural Information Processing Systems*, volume 32, 2019. URL https://proceedings.neurips.cc/paper/2019/hash/6fb52e71b837628ac16539c1ff911667-Abstract.html.

Bram Bakker. Reinforcement learning with long short-term memory. In *Advances in Neural Information Processing Systems*, volume 14, pages 1475–1482, 2001. URL https://mlanthology.org/neurips/2001/bakker2001neurips-reinforcement/.

Elias Bareinboim, Juan D. Correa, Duligur Ibeling, and Thomas Icard. On Pearl's hierarchy and the foundations of causal inference. In *Probabilistic and Causal Inference: The Works of Judea Pearl*, pages 507–556. ACM, 2022. doi: 10.1145/3501714.3501743.

André Barreto, Will Dabney, Rémi Munos, Jonathan J. Hunt, Tom Schaul, Hado P. van Hasselt, and David Silver. Successor features for transfer in reinforcement learning. In *Advances in Neural Information Processing Systems*, volume 30, pages 4055–4065, 2017. URL https://proceedings.neurips.cc/paper/2017/hash/350db081a661525235354dd3e19b8c05-Abstract.html.

Jacob D. Bekenstein. Universal upper bound on the entropy-to-energy ratio for bounded systems. *Physical Review D*, 23(2):287–298, 1981. doi: 10.1103/PhysRevD.23.287. URL https://doi.org/10.1103/PhysRevD.23.287.

Michael Timothy Bennett. The optimal choice of hypothesis is the weakest, not the shortest. In *16th International Conference on Artificial General Intelligence*, Lecture Notes in Computer Science, pages 42–51. Springer, 2023a. doi: 10.1007/978-3-031-33469-6_5. URL https://doi.org/10.1007/978-3-031-33469-6_5.

Michael Timothy Bennett. Emergent causality and the foundation of consciousness. In *16th International Conference on Artificial General Intelligence*, Lecture Notes in Computer Science, pages 52–61. Springer, 2023b. doi: 10.1007/978-3-031-33469-6_6. URL https://doi.org/10.1007/978-3-031-33469-6_6.

Michael Timothy Bennett. Is complexity an illusion? In *17th International Conference on Artificial General Intelligence*, Lecture Notes in Computer Science. Springer, 2024a. doi: 10.1007/978-3-031-65572-2_2. URL https://doi.org/10.1007/978-3-031-65572-2_2.

Michael Timothy Bennett. Are biological systems more intelligent than artificial intelligence?, 2024b. URL https://arxiv.org/abs/2405.02325. In press, Philosophical Transactions of the Royal Society B: Biological Sciences. Special issue on Hybrid agencies: crossing borders between biological and artificial worlds.

Michael Timothy Bennett. *How To Build Conscious Machines*. PhD thesis, The Australian National University, 2025. URL https://hdl.handle.net/1885/733782452.

Pablo Samuel Castro. Scalable methods for computing state similarity in deterministic Markov decision processes. *Proceedings of the AAAI Conference on Artificial Intelligence*, 34(06):10069–10076, 2020.

Dian Chen, Brady Zhou, Vladlen Koltun, and Philipp Krähenbühl. Learning by cheating. In *Proceedings of the Conference on Robot Learning*, volume 100 of *Proceedings of Machine Learning Research*, pages 66–75, 2020. URL https://proceedings.mlr.press/v100/chen20a.html.

Norman Ferns, Prakash Panangaden, and Doina Precup. Metrics for finite Markov decision processes. In *Proceedings of the 20th Conference on Uncertainty in Artificial Intelligence*, pages 162–169. AUAI Press, 2004.

Norman Ferns, Prakash Panangaden, and Doina Precup. Bisimulation metrics for continuous Markov decision processes. *SIAM Journal on Computing*, 40(6):1662–1714, 2011. doi: 10.1137/10080484X.

Justin Fu, Mohammad Norouzi, Ofir Nachum, George Tucker, Ziyu Wang, Alexander Novikov, Mengjiao Yang, Michael R. Zhang, Yutian Chen, Aviral Kumar, Cosmin Paduraru, Sergey Levine, and Thomas Paine. Benchmarks for deep off-policy evaluation. In *International Conference on Learning Representations*, 2021. URL https://mlanthology.org/iclr/2021/fu2021iclr-benchmarks/.

Saurabh Garg, Sivaraman Balakrishnan, Zachary Chase Lipton, Behnam Neyshabur, and Hanie Sedghi. Leveraging unlabeled data to predict out-of-distribution performance. In *International Conference on Learning Representations*, 2022. URL https://mlanthology.org/iclr/2022/garg2022iclr-leveraging/.

Carles Gelada, Saurabh Kumar, Jacob Buckman, Ofir Nachum, and Marc G. Bellemare. DeepMDP: Learning continuous latent space models for representation learning. In *Proceedings of the 36th International Conference on Machine Learning*, volume 97 of *Proceedings of Machine Learning Research*, pages 2170–2179. PMLR, 2019.

Robert Givan, Thomas Dean, and Matthew Greig. Equivalence notions and model minimization in Markov decision processes. *Artificial Intelligence*, 147(1-2):163–223, 2003.

Dan Hendrycks and Kevin Gimpel. A baseline for detecting misclassified and out-of-distribution examples in neural networks. In *International Conference on Learning Representations*, 2017. URL https://mlanthology.org/iclr/2017/hendrycks2017iclr-baseline/.

John Hewitt and Percy Liang. Designing and interpreting probes with control tasks. In *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing and the 9th International Joint Conference on Natural Language Processing (EMNLP-IJCNLP)*, pages 2733–2743, Hong Kong, China, 2019. Association for Computational Linguistics. doi: 10.18653/v1/D19-1275. URL https://aclanthology.org/D19-1275/.

Geoffrey Hinton, Oriol Vinyals, and Jeff Dean. Distilling the knowledge in a neural network, 2015. URL https://arxiv.org/abs/1503.02531.

Max Jaderberg, Volodymyr Mnih, Wojciech Marian Czarnecki, Tom Schaul, Joel Z. Leibo, David Silver, and Koray Kavukcuoglu. Reinforcement learning with unsupervised auxiliary tasks. In *International Conference on Learning Representations*, 2017. URL https://mlanthology.org/iclr/2017/jaderberg2017iclr-reinforcement/.

Yiding Jiang, Behnam Neyshabur, Hossein Mobahi, Dilip Krishnan, and Samy Bengio. Fantastic generalization measures and where to find them. In *International Conference on Learning Representations*, 2020.

Yiding Jiang, Vaishnavh Nagarajan, Christina Baek, and J. Zico Kolter. Assessing generalization of SGD via disagreement. In *International Conference on Learning Representations*, 2022. URL https://mlanthology.org/iclr/2022/jiang2022iclr-assessing/.

Leslie Pack Kaelbling, Michael L. Littman, and Anthony R. Cassandra. Planning and acting in partially observable stochastic domains. *Artificial Intelligence*, 101(1-2):99–134, 1998.

Sham Kakade and John Langford. Approximately optimal approximate reinforcement learning. In *Proceedings of the Nineteenth International Conference on Machine Learning*, pages 267–274. Morgan Kaufmann, 2002. URL https://dl.acm.org/doi/10.5555/645531.656005.

Polina Kirichenko, Pavel Izmailov, and Andrew Gordon Wilson. Last layer re-training is sufficient for robustness to spurious correlations. In *International Conference on Learning Representations*, 2023. URL https://mlanthology.org/iclr/2023/kirichenko2023iclr-last/.

Jan Leike and Marcus Hutter. Bad universal priors and notions of optimality. In Peter Gr"unwald, Elad Hazan, and Satyen Kale, editors, *Proceedings of The 28th Conference on Learning Theory*, volume 40 of *Proceedings of Machine Learning Research*, pages 1244–1259, Paris, France, 2015. PMLR. doi: 10.48550/arxiv.1510.04931. URL https://proceedings.mlr.press/v40/Leike15.html.

Lihong Li, Thomas J. Walsh, and Michael L. Littman. Towards a unified theory of state abstraction for MDPs. In *International Symposium on Artificial Intelligence and Mathematics*, 2006.

Steven Morad, Ryan Kortvelesy, Matteo Bettini, Stephan Liwicki, and Amanda Prorok. POPGym: Benchmarking partially observable reinforcement learning. In *International Conference on Learning Representations*, 2023.

Pietro Morerio, Jacopo Cavazza, and Vittorio Murino. Minimal-entropy correlation alignment for unsupervised deep domain adaptation. In *International Conference on Learning Representations*, 2018. URL https://mlanthology.org/iclr/2018/morerio2018iclr-minimalentropy/.

Aran Nayebi. What capable agents must know: Selection theorems for robust decision-making under uncertainty. In *Proceedings of the 42nd Conference on Uncertainty in Artificial Intelligence*, volume 337 of *Proceedings of Machine Learning Research*, pages 4773–4795. PMLR, 2026. URL https://proceedings.mlr.press/v337/nayebi26a.html.

Tianwei Ni, Benjamin Eysenbach, and Ruslan Salakhutdinov. Recurrent model-free RL can be a strong baseline for many POMDPs. In *Proceedings of the 39th International Conference on Machine Learning*, volume 162 of *Proceedings of Machine Learning Research*, pages 16691–16723. PMLR, 2022. URL https://proceedings.mlr.press/v162/ni22a.html.

Tianwei Ni, Michel Ma, Benjamin Eysenbach, and Pierre-Luc Bacon. When do transformers shine in RL? decoupling memory from credit assignment. In *Advances in Neural Information Processing Systems*, volume 36, 2023.

Tom Le Paine, Cosmin Paduraru, Andrea Michi, Caglar Gulcehre, Konrad Żołna, Alexander Novikov, Ziyu Wang, and Nando de Freitas. Hyperparameter selection for offline reinforcement learning, 2020. URL https://arxiv.org/abs/2007.09055.

Emilio Parisotto, Francis Song, Jack Rae, Razvan Pascanu, Caglar Gulcehre, Siddhant Jayakumar, Max Jaderberg, Raphaël Lopez Kaufman, Aidan Clark, Seb Noury, Matthew Botvinick, Nicolas Heess, and Raia Hadsell. Stabilizing transformers for reinforcement learning. In *Proceedings of the 37th International Conference on Machine Learning*, volume 119 of *Proceedings of Machine Learning Research*, pages 7487–7498, 2020. URL https://proceedings.mlr.press/v119/parisotto20a.html.

Judea Pearl. *Causality: Models, Reasoning, and Inference*. Cambridge University Press, Cambridge, 2 edition, 2009.

Tiago Pimentel, Josef Valvoda, Rowan Hall Maudslay, Ran Zmigrod, Adina Williams, and Ryan Cotterell. Information-theoretic probing for linguistic structure. In *Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics*, pages 4609–4622, Online, 2020. Association for Computational Linguistics. doi: 10.18653/v1/2020.acl-main.420. URL https://aclanthology.org/2020.acl-main.420/.

Lerrel Pinto, Marcin Andrychowicz, Peter Welinder, Wojciech Zaremba, and Pieter Abbeel. Asymmetric actor critic for image-based robot learning. In *Proceedings of Robotics: Science and Systems XIV*, 2018. doi: 10.15607/RSS.2018.XIV.008. URL https://www.roboticsproceedings.org/rss14/p08.html.

Jonathan Richens and Tom Everitt. Robust agents learn causal world models. In *The Twelfth International Conference on Learning Representations*, 2024. doi: 10.48550/arxiv.2402.10877. URL https://arxiv.org/abs/2402.10877.

Jonathan Richens, Tom Everitt, and David Abel. General agents need world models. In *Proceedings of the 42nd International Conference on Machine Learning*, volume 267 of *Proceedings of Machine Learning Research*, pages 51659–51687. PMLR, 2025. URL https://proceedings.mlr.press/v267/richens25a.html.

John Schulman, Filip Wolski, Prafulla Dhariwal, Alec Radford, and Oleg Klimov. Proximal policy optimization algorithms. *arXiv preprint arXiv:1707.06347*, 2017. doi: 10.48550/arXiv.1707.06347. URL https://arxiv.org/abs/1707.06347.

Trey Smith and Reid G. Simmons. Heuristic search value iteration for POMDPs. In *Proceedings of the 20th Conference on Uncertainty in Artificial Intelligence*, pages 520–527, 2004. URL https://mlanthology.org/uai/2004/smith2004uai-heuristic/.

Masashi Sugiyama, Matthias Krauledat, and Klaus-Robert Müller. Covariate shift adaptation by importance weighted cross validation. *Journal of Machine Learning Research*, 8:985–1005, 2007. URL https://jmlr.org/papers/v8/sugiyama07a.html.

Richard S Sutton and Andrew G Barto. *Reinforcement learning: An introduction*. MIT press, MA, 2018.

Ruo Yu Tao, Kaicheng Guo, Cameron Allen, and George Konidaris. Benchmarking partial observability in reinforcement learning with a suite of memory-improvable domains. In *Proceedings of the Second Reinforcement Learning Conference*, 2025.

Ahmed Touati and Yann Ollivier. Learning one representation to optimize all rewards. In *Advances in Neural Information Processing Systems*, volume 34, pages 13–23, 2021. URL https://proceedings.neurips.cc/paper/2021/hash/003dd617c12d444ff9c80f717c3fa982-Abstract.html.

Yilun Xu, Shengjia Zhao, Jiaming Song, Russell Stewart, and Stefano Ermon. A theory of usable information under computational constraints. In *International Conference on Learning Representations*, 2020. URL https://mlanthology.org/iclr/2020/xu2020iclr-theory/.

Kaichao You, Ximei Wang, Mingsheng Long, and Michael Jordan. Towards accurate model selection in deep unsupervised domain adaptation. In *Proceedings of the 36th International Conference on Machine Learning*, volume 97 of *Proceedings of Machine Learning Research*, pages 7124–7133, 2019. URL https://mlanthology.org/icml/2019/you2019icml-accurate/.

Amy Zhang, Rowan McAllister, Roberto Calandra, Yarin Gal, and Sergey Levine. Learning invariant representations for reinforcement learning without reconstruction. In *International Conference on Learning Representations*, 2021.